"EVENT MANAGEMENT SYSTEM"

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

KUNAMANENI BHAVANA	**2520030003**
KODEBOINA PRIYADARSHINI	**2520030067**
CHEETI SRIHITHA RAO	**2520030400**

Under the Guidance of

Dr. S. Vinay Kumar
Associate Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in
2026-27

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled **"EVENT MANAGEMENT SYSTEM"** is a record of Bonafide work of **K.BHAVANA– 2520030003, K.PRIYADARSHINI– 2520030067**, **CH.SRIHITHA– 2520030400** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The results embodied in this report have not been copied from any other departments/University/Institute.

K.Bhavana– 2520030003

K.Priyadharshini– 2520030067

CH.Srihitha– 2520030400

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is certify that the mini project based report entitled **"EVENT MANAGEMENT SYSTEM"** is a bonafide work done and submitted by **K.BHAVANA-2520030003, K.PRIYADARSHINI-2520030067, CH.SRIHITHA-2520030400**in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science Engineering, during the academic year 2026-27.

Signature of the Guide
(Dr. S. Vinay Kumar)

Signature of the HOD

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who supported and guided us in the successful completion of our project, **Event Management System**. This project has been a valuable learning experience that helped us improve our technical knowledge, programming skills, and understanding of software development. The completion of this project would not have been possible without the valuable guidance, encouragement, and support we received throughout its development.

We express our heartfelt thanks to our project guide, **Dr. S. Vinay Kumar, Associate Professor, Department of Computer Science and Engineering**, for his continuous guidance, valuable suggestions, encouragement, and support throughout the project. His guidance helped us understand the project requirements, organize our work effectively, and improve our approach to designing and implementing the system. We are grateful for the time and effort he dedicated to guiding us at every stage of the project.

We also extend our sincere gratitude to **Dr. P. Venkateshwara Rao, Associate Professor and Head of the Department of Computer Science and Engineering**, for providing us with the necessary support, encouragement, and academic environment to complete this project successfully. Finally, we thank our faculty members, classmates, and everyone who directly or indirectly contributed to the successful completion of this project. Their support and encouragement have been valuable throughout this learning journey.

K.Bhavana– 2520030003

K.Priyadarshini– 2520030067

CH.Srihitha– 2520030400

Abstract

Colleges, universities, and organizations conduct various events such as seminars, workshops, hackathons, conferences, technical fests, and cultural programs. Managing these events manually can result in scheduling conflicts, duplicate registrations, communication gaps, and inefficient data management. The proposed Event Management System (EMS) is a web-based platform designed to automate event creation, searching, registration, scheduling, resource allocation, and reporting by integrating advanced algorithmic techniques. The system uses string algorithms such as KMP and Rabin-Karp for efficient event and participant searching. Dynamic Programming, particularly Edit Distance, is used for fuzzy search and correcting user queries. Suffix Arrays and LCP can support efficient similarity analysis of event descriptions and documents. Network Flow algorithms such as Dinic's algorithm are applied to resource allocation, participant assignment, and conflict-free scheduling. For complex scheduling and allocation problems, NP-hard concepts and approximation algorithms such as the Vertex Cover 2-approximation provide practical solutions when exact solutions are computationally expensive. Randomized algorithms such as Miller-Rabin primality testing and randomized hashing demonstrate efficient probabilistic processing, while parallel algorithms such as parallel reduction and prefix-sum can improve the processing of large event datasets. By combining these techniques, the EMS provides efficient searching, scheduling, allocation, data processing, and management, reducing administrative workload and providing a reliable and user-friendly experience for organizers and participants.

Table of Contents

	Acknowledgement		Page no
	Abstract		6
	Table of Contents		7
	List of Figures		8
1	Introduction	1.1 Overview 1.2 Problem statement 1.3 Scope of the project 1.4 Objectives	9-12
2	System Requirements	2.1 Functional Requirements 2.2Non-Functional Requirements	13-28
3	System Design	3.1 High-level Architecture Diagram 3.2Description of each layer	29-33
4	Implementation	4.1 Module-wise implementation	34-42
5	Results		43-46
6	Challenges & Limitations	6.1 Issues faced during development	47-53
7	Conclusion & Future Work	7.1 Conclusion 7.2 Future Work 7.3 Expected Outcome Of Futur Enhancements	54-60

References Books, tutorials, APIs, documentation sites used 61

List of Figures

1.1	OVERVIEW	9
1.2	PROBLEM STATEMENT	10
1.3	SCOPE OF THE PROJECT	10-11
1.4	OBJECTIVE	11-12
2.1	FUNCTIONAL REQUIREMENTS	14-22
2.2	NON - FUNCTIONAL REQUIREMENTS	22-28
3.1	HIGH-LEVEL ARCHITECTURE DIAGRAM	29
3.2	DESCRIPTION OF EACH LAYER	30-33
4.1	MODULE-WISE IMPLEMENTATION	34-40
4.2	USER INTERFACE AND MENU LOGIC	40
4.3	EVENT MANAGEMENT	40
4.4	PARTICIPATE AND VOLUNTEER MANAGEMENT	41
4.5	ALGORITHM INTEGRATION AND COMPLEXITY	41-43

5.1 RESULTS SCREENSHOT 43-45

5.2 FUNCTIONAL TESTING 46

6.1 CHALLENGES FACED DURING DEVELOPMENT 47-50

6.2 LIMITATIONS OF THE CURRENT SYSTEM 50-53

6.3 METHODS FOR REDUCING THE CHALLENGES 53-54

7.1 CONCLUSION 54-55

7.2 FUTURE WORK 56-60

7.3 EXPECTED OUTCOME OF FUTURE ENHANCEMENTS 60

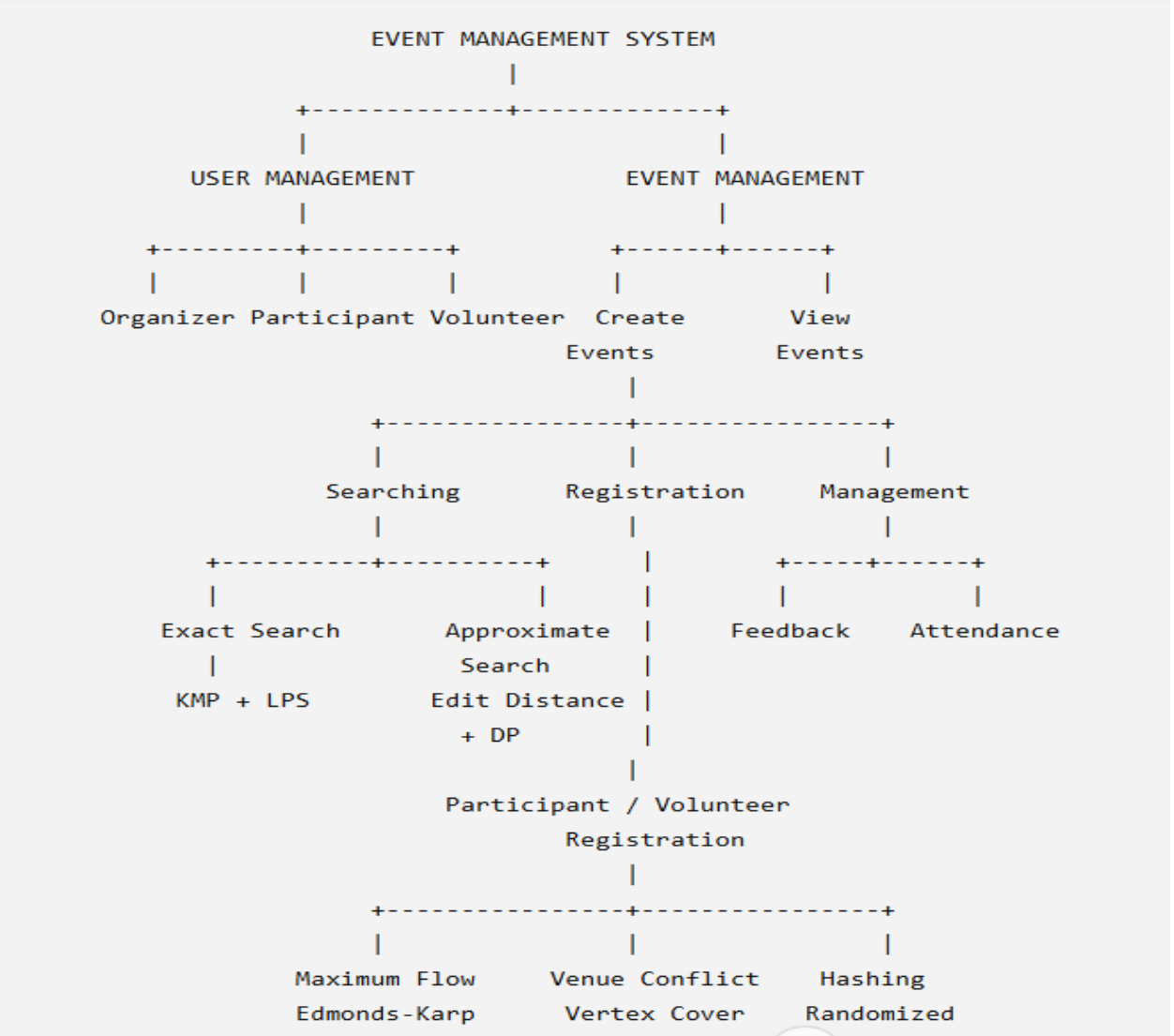
1. INTRODUCTION

1.1 Overview

The **Event Management System (EMS)** is a Java-based application developed to simplify and organize the activities involved in planning and managing events. The system provides a common platform through which different users can interact with events according to their roles. The major user roles supported by the system are **Organizer, Participant, and Volunteer**.

A major feature of the proposed Event Management System is the integration of **Data Structures and Algorithms into practical event-management operations**. Instead of treating algorithms as independent programs, the system maps different algorithms to specific problems encountered in event management. The major algorithms incorporated into the system include **KMP with LPS, Edit Distance with Dynamic Programming, Randomized Hashing, Edmonds-Karp Maximum Flow, and Vertex Cover 2-Approximation**.

Efficient algorithms help improve information retrieval and the processing of data in software applications [10].



1.2 Problem Statement

Managing multiple events manually becomes challenging as the number of organizers, participants, volunteers, registrations, and event activities increases. Conventional event-management methods may make it difficult to efficiently search event information, manage registrations, handle participant-event assignments, maintain attendance, collect feedback, and identify conflicts between events. Searching also becomes a challenge when users do not enter the exact event name or provide a slightly different search term.

Another important problem is the effective allocation of participants to events while considering event capacities. Similarly, when multiple events use the same venue, conflicts may occur and need to be identified systematically. Managing these activities separately without appropriate algorithms can lead to inefficient processing and difficulty in maintaining organized records.

Therefore, there is a need for an **Event Management System** that combines event-management operations with appropriate Data Structures and Algorithms. The proposed

system provides role-based functionality for **Organizers, Participants, and Volunteers** and supports event creation, event searching, registration, volunteering, feedback, and attendance management. It incorporates **KMP with LPS for exact event searching, Edit Distance with Dynamic Programming for approximate searching, Randomized Hashing for search filtering, Edmonds-Karp/Maximum Flow for participant-event assignment, and Vertex Cover 2-Approximation for event venue-conflict analysis**. This integration provides a systematic approach to solving practical event-management problems while demonstrating the application of algorithmic techniques in a real-world system.

The application supports three principal user roles: Organizer, Participant, and Volunteer. Organizers are responsible for creating events and maintaining event-related information. Participants can view available events, search for suitable events, register, submit feedback, and view attendance information. Volunteers can view events, search for opportunities, register as volunteers, and review their volunteering activities. Role-based menus ensure that users are presented with operations relevant to their responsibilities, making the application easier to understand and operate.

The Event Management System (EMS) is a Java-based application developed to simplify the planning, organization, and management of events. Colleges, universities, student associations, and other organizations conduct many types of events, including seminars, workshops, technical competitions, cultural programmes, conferences, and sports activities. Each event requires coordination of information such as its title, description, date, time, venue, organizer, capacity, registrations, volunteers, attendance, and participant feedback. When these activities are handled manually or through separate records, information can become difficult to maintain and retrieve. EMS provides a common application through which these event-related activities can be managed in a structured manner.

Approximate string-matching methods can be used to identify similarities between strings even when they are not identical [3], [8].

1.3 Scope of the Project

The scope of the **Event Management System (EMS)** covers the development of a Java-based application for organizing and managing events involving **Organizers, Participants, and Volunteers**. The system provides role-based functionality for creating, viewing, searching, and participating in events. Organizers can create events, manage attendance, view feedback, and perform participant-event assignment analysis. Participants can view and search events, register for events, provide feedback, and view their attendance records. Volunteers can view available events, search for events, register as volunteers, and view their volunteering activities.

The project also focuses on applying **Data Structures and Algorithms to practical event-management problems**. KMP with LPS is used for exact event searching, while Edit Distance with Dynamic Programming supports approximate searching when the entered search term differs from the stored event information. Randomized Hashing is integrated with event searching. Edmonds-Karp and Maximum Flow are used for participant-event assignment, while Vertex Cover 2-Approximation is applied to identify conflicts between events sharing the same venue.

The system also includes file-based storage for users, events, registrations, volunteers, feedback, and attendance. Overall, the project demonstrates how algorithmic techniques can be integrated with event-management operations to provide an organized and efficient system.

Event discovery is another challenge. Users may not remember the complete event title or may enter a search term containing a spelling variation. A search process that accepts only an exact title can fail to return relevant information even when the desired event exists. Therefore, the system needs both exact searching and approximate searching so that users can locate events more effectively.

Managing several events at the same time involves more than announcing an event. Organizers must maintain accurate event details, provide information to users, handle registrations, coordinate volunteers, monitor attendance, collect feedback, and ensure that available capacity is not exceeded. As the number of events and users increases, maintaining these records manually can require considerable time and effort. Records stored in different places may become inconsistent, and retrieving the correct information can be difficult.

1.4 Objectives

The main objective of the **Event Management System (EMS)** is to develop an organized Java-based application that simplifies event-related activities while demonstrating the practical application of Data Structures and Algorithms. The specific objectives are:

To provide role-based access for Organizers, Participants, and Volunteers, allowing each user to perform activities according to their responsibilities.

To manage events efficiently by providing facilities for event creation, viewing, registration, volunteering, feedback, and attendance management.

To implement efficient event searching using the **KMP algorithm with LPS** for exact pattern matching.

To support approximate event searching using **Edit Distance and Dynamic Programming**, allowing users to find relevant events even when search terms are slightly different.

To integrate Randomized Hashing into the event-searching process for efficient search filtering and matching.

To apply Edmonds-Karp and Maximum Flow techniques to model and analyze participant-event assignments while considering event capacities.

To identify venue conflicts between events using a graph representation and **Vertex Cover 2-Approximation**.

To maintain event-related information persistently through file-based storage for users, events, registrations, volunteers, feedback, and attendance.

To demonstrate the practical application of DSA concepts by connecting algorithmic techniques with real-world event-management requirements.

The project also includes file handling for maintaining user details, event records, registrations, volunteer records, feedback, and attendance. The system validates important inputs and checks conditions such as duplicate registrations, event capacity, valid feedback ratings, and Present/Absent attendance values. The current implementation is a Java-based, menu-driven application with local file storage; it does not claim to provide a web or mobile interface, a relational database, or automatic booking through the flow-analysis module. These features may be considered for future development.

The algorithmic scope includes exact and approximate event searching, assignment analysis, and venue-conflict analysis. KMP with LPS is used for exact pattern matching. Edit Distance with Dynamic Programming calculates the number of insertions, deletions, and substitutions needed to transform one string into another and supports approximate matching. Randomized hashing is integrated as a search-filtering technique. Edmonds–Karp maximum flow models participant–event assignments while considering event capacities, and Vertex Cover 2-Approximation is applied to a graph representing events that share venues.

2. SYSTEM REQUIREMENTS

The System Requirements section describes the functional capabilities and quality characteristics required for the successful development and operation of the Event Management System. System requirements define what the application must accomplish, how users interact with its features, how information is maintained, and how the system should behave during normal and exceptional situations. Clearly defining these requirements helps establish the foundation for system design, implementation, testing, and evaluation.

The Event Management System (EMS) is a Java-based application developed to simplify and organize the activities associated with planning, conducting, and managing events. Events such as technical symposiums, workshops, seminars, conferences, cultural festivals, competitions, and academic programs involve several activities, including event creation, participant registration, volunteer coordination, attendance recording, and feedback collection. When

these activities are managed independently or manually, organizers may experience difficulties maintaining accurate records and coordinating information.

The proposed system provides a common platform through which Organizers, Participants, and Volunteers can perform their respective activities. Each user category has a separate set of functions based on its responsibilities. Organizers can create and manage events, while Participants can search for events, register for suitable activities, provide feedback, and view attendance information. Volunteers can explore available events, register for volunteering activities, and view their associated records. This role-based approach helps maintain an organized workflow and ensures that users can access the functions relevant to their responsibilities.

In addition to providing event-management features, the system demonstrates the practical application of Data Structures and Algorithms (DSA). Event searching is supported by the Knuth–Morris–Pratt (KMP) algorithm with the Longest Prefix Suffix (LPS) array, while approximate searching is supported by Edit Distance using Dynamic Programming. Randomized Hashing is integrated into the searching process as a filtering technique. Edmonds–Karp Maximum Flow is used to analyze participant-event assignments while considering event capacities. Vertex Cover 2-Approximation is applied to analyze conflicts between events that share venues.

The system also uses file-based storage to maintain information about users, events, registrations, volunteers, feedback, and attendance. This allows stored records to be retrieved during subsequent program executions without requiring a separate database for the current implementation. The requirements described in this chapter provide a structured explanation of the features and quality expectations of the application.

The system requirements are divided into two major categories: functional requirements and non-functional requirements. Functional requirements explain the operations that the system must perform, whereas non-functional requirements describe the expected qualities of the application, including usability, performance, reliability, data integrity, security, maintainability, scalability, accuracy, persistence, error handling, portability, and algorithmic efficiency.

The system's search functionality can be designed using established string-matching techniques [1], [4].

2.1 Functional Requirements

The functional requirements specify the operations that the **Event Management System (EMS)** must perform. The system provides different functionalities for **Organizers, Participants, and Volunteers** and integrates DSA algorithms for event searching, assignment, and conflict analysis.

The functional requirements of the proposed system are designed around the complete event-management process. These requirements cover user registration and login, role-based access, event creation, event viewing, event searching, participant registration, volunteer management, participant-event assignment analysis, venue-conflict analysis, feedback collection, attendance management, and file-based data handling.

Each function is implemented through appropriate Java methods and interacts with the relevant data structures, algorithms, and stored records. The following subsections explain the major functional requirements in detail.

1. User Registration and Login

The system shall provide registration and login facilities to identify users before they access role-specific operations. Registration allows a new user to create an account by providing the required details and selecting an appropriate role. The supported roles are Organizer, Participant, and Volunteer.

During registration, the application collects the information required by its account-management functionality and stores the user record for future use. The registration process establishes the relationship between the user and the activities that the user can perform within the application. Appropriate validation should be performed to identify incomplete information and duplicate usernames where applicable.

The login function allows an existing user to access the system using registered credentials. The application checks the supplied credentials against the stored user information. If the credentials are valid, the user is permitted to continue to the appropriate menu. If the credentials are invalid, the system should display a suitable message and allow the user to try again.

After successful authentication, the application identifies the user's role and displays the corresponding menu. An Organizer receives options related to event creation, management, and analysis. A Participant receives options for viewing events, searching, registration, feedback, and attendance. A Volunteer receives options for viewing events, searching, volunteer registration, and reviewing volunteering activities.

This functionality separates the operations associated with different user categories and provides an organized interaction process. It also ensures that users do not need to navigate through unrelated menu options when performing their tasks.

2. Role-Based Access and Menu Management

The Event Management System shall provide different menus according to the role selected during registration and stored in the user's account. Role-based menu management organizes the application into separate functional areas and makes the interface easier to understand.

The Organizer menu provides access to event creation and management operations. It supports the entry of event details, viewing available events, performing event searches, examining participant-event assignments, analysing venue conflicts, and managing attendance and feedback-related activities.

The Participant menu focuses on event discovery and participation. Participants can view event information, search for events by entering relevant terms, register for selected events, review their registrations, submit feedback, and view attendance information.

The Volunteer menu provides facilities for discovering events and registering to assist with their organization. Volunteers can view available events, search for suitable opportunities, register for volunteering activities, and view their volunteering records.

The system shall display clear menu options and guide users through the sequence of operations. When a user selects an option, the application should execute the corresponding method and display the result. After completing an operation, the user should be able to return to the appropriate menu and continue using the application.

Role-based menus improve the organization of the system by grouping related operations together. However, the presence of separate menus alone does not constitute a complete security mechanism; stronger authorization checks would be required in a future implementation.

3. Event Creation and Management

The system shall allow authorized Organizers to create new events by entering the required event information. Event creation is one of the primary functions because other operations, such as event searching, participant registration, volunteering, and attendance management, depend on the availability of event records.

The event information maintained by the application includes the event name, description, date, day, start time, end time, venue, organizer, and capacity. These details help users understand the purpose, schedule, location, and participation limits of an event.

When an Organizer creates an event, the application shall accept the supplied information, validate the required fields, and create a record containing the event details. A valid event identifier should be maintained wherever required by the implementation so that the event can be referenced during registration, attendance recording, and feedback collection.

The system should check important input conditions before accepting event information. For example, event capacity should be a valid value, and dates and times should follow the expected format. Missing or invalid information should be handled through suitable messages rather than being accepted without verification.

Once an event has been created, its information shall be stored in the corresponding file. The system should make the stored event available for subsequent viewing and searching operations. This ensures that the application can maintain an organized collection of events instead of treating each event as temporary information entered during a single operation.

Event-management functionality provides the foundation for the other modules. A correctly maintained event record allows participants and volunteers to identify suitable activities and enables organizers to analyse participation and venue-related conditions.

4. Viewing Available Events

The system shall allow users to view the events available in the application. This function helps Participants and Volunteers identify activities that match their interests, while Organizers can use it to review the events maintained by the system.

The application shall retrieve the relevant event records from its file-based storage and display the available information in a readable format. Depending on the stored fields, the displayed information may include the event identifier, event name, description, date, day, start time, end time, venue, organizer, and capacity.

Presenting the information in a consistent format makes it easier for users to compare events and understand their schedules. It also helps participants identify the events for which they may wish to register and allows volunteers to discover opportunities that require their involvement.

The system should handle situations in which no event records are available. Instead of displaying confusing or incomplete information, it should inform the user that no events are currently available. Similarly, if event information cannot be retrieved because of a file-related problem, the application should provide an appropriate error message.

The viewing function acts as a connection between event creation and event participation. Events entered by Organizers become available for discovery, while the stored event details provide the information required by registration, searching, and other event-management operations.

5. Event Searching Using KMP and LPS

The system shall provide an event-searching facility that allows users to locate relevant events by entering a search term. Searching is important because the amount of event information may increase as organizers create more events. Examining every event manually can become inconvenient, particularly when users are interested in a specific event name or keyword.

The Knuth–Morris–Pratt (KMP) algorithm is used to perform exact pattern matching. It searches for a given pattern within event information and avoids unnecessary character comparisons by using information obtained during preprocessing. The algorithm constructs an LPS array, where LPS stands for Longest Proper Prefix which is also a Suffix.

The LPS array helps the searching procedure determine how far the pattern can be shifted after a mismatch. Instead of restarting the comparison from the beginning of the pattern every time a mismatch occurs, KMP reuses previously established matching information. This improves the efficiency of pattern searching.

For example, if a user searches for the term Workshop, the system can examine stored event information for the occurrence of that pattern. When the pattern matches the relevant event information, the corresponding event can be included in the search results.

The KMP algorithm has a time complexity of $O(n+m)$, where n represents the length of the text being searched and m represents the length of the pattern. Its preprocessing stage takes $O(m)$ time, and the subsequent search takes $O(n)$ time under the standard algorithm.

The application shall accept search input, process the pattern using the implemented searching logic, and display matching event information. If no exact match is found, it should inform the user and allow another search to be performed.

This requirement demonstrates the application of string-matching techniques to event discovery. It also helps users locate relevant information without manually examining every stored event.

6. Approximate Event Searching Using Edit Distance

In addition to exact searching, the system shall support approximate event searching through Edit Distance using Dynamic Programming. Exact searching may fail when users enter a term containing spelling errors, missing characters, additional characters, or small differences from the stored event information.

Edit Distance measures the minimum number of editing operations required to transform one string into another. The standard operations include insertion, deletion, and substitution of characters. Dynamic Programming is used to calculate the minimum editing cost by storing and reusing the results of smaller subproblems.

For example, a user may enter Hackaton while the stored event name is Hackathon. An exact search may not identify the intended event because the two strings are different. An approximate-searching method can calculate their edit distance and identify that the terms are closely related.

The system shall compare the entered search term with relevant stored event information and use the calculated distance to determine whether a candidate is sufficiently similar. The accepted distance threshold depends on the searching logic implemented in the application.

In the standard Dynamic Programming approach, a table is constructed in which each cell represents the minimum editing cost for transforming a prefix of one string into a prefix of another string. The values are calculated by considering insertion, deletion, and substitution operations. The final table entry gives the edit distance between the complete strings.

The standard time and space complexities of this approach are $O(mn)$, where m and n are the lengths of the two strings, respectively.

Approximate searching makes the system more flexible because users do not always remember the exact spelling or complete name of an event. It helps identify potentially relevant results even when the entered query differs slightly from the stored information. The application should display the relevant matching events and inform the user when no suitable result is found.

7. Randomized Hashing for Search Filtering

The system shall integrate Randomized Hashing into the event-searching process as a filtering mechanism. Hashing converts a sequence of characters into a numerical value that can be used to compare candidate patterns efficiently.

In the context of event searching, a hash value can be calculated for the entered search pattern and relevant portions of the stored event information. When the calculated values match, the corresponding text can be treated as a candidate for further verification

A hash match alone does not guarantee that two strings are identical because different strings may produce the same hash value. Such a situation is called a hash collision. Therefore, the system must verify candidate matches using character comparisons or the appropriate exact-matching procedure.

Randomized or rolling-hash techniques can reduce the amount of repeated processing required when examining multiple candidate positions. The precise efficiency depends on the hash implementation, the selected parameters, and the input data.

For example, when a user searches for a keyword in event descriptions, hashing can help identify possible matching sections before the system performs the required verification. This approach demonstrates how hashing techniques can be integrated with string-searching algorithms.

The system shall use hashing as a supporting mechanism rather than treating hash equality as conclusive proof of a match. This requirement helps maintain search accuracy while demonstrating the practical use of randomized algorithmic techniques.

8. Participant Registration for Events

The system shall allow Participants to register for events available in the application. Registration establishes the relationship between a participant and a selected event and enables the system to maintain records of participation.

Before accepting a registration request, the application shall verify that the selected event exists. It should also check whether the participant has already registered for the same event. If a duplicate registration is detected, the application shall reject the repeated request and display a suitable message.

The system shall check the event's capacity before accepting a new registration. If the event has reached its permitted capacity, the registration should not be accepted as a successful new registration. If capacity is available and the other validation conditions are satisfied, the application can store the registration record.

The registration record should contain the information required to identify the participant and the selected event. The record must be stored so that the participant's registrations can be retrieved during later operations.

Participants should be able to view their registered events. This function helps them review their participation choices and obtain information about the events they have selected.

Registration management is closely related to data integrity because duplicate registrations and capacity violations can produce inaccurate participation records. Appropriate validation helps ensure that the stored information reflects accepted registration requests.

The application also uses participant-event assignment analysis to demonstrate Maximum Flow techniques. This analysis is distinct from automatically accepting registrations: the flow calculation evaluates possible assignments based on the network and capacities represented in the implementation.

9. Volunteer Registration and Management

The system shall provide separate volunteer-management functionality for users registered with the Volunteer role. Volunteers contribute to the organization of events by assisting with activities and supporting event-related responsibilities.

The application shall allow Volunteers to view available events and search for events that may require their involvement. After identifying a suitable event, a Volunteer can register for the selected volunteering opportunity through the available menu option.

Before storing a volunteer registration, the system shall check whether the relevant record already exists. Duplicate volunteer registrations for the same event should be prevented to maintain accurate records and avoid repeated entries.

The system shall store volunteer information separately from participant registration information so that the two activities can be managed independently. This separation helps distinguish individuals participating in an event from individuals registering to volunteer for its organization.

Volunteers should be able to view the events associated with their volunteering activities. The application should retrieve the corresponding records and present them in an understandable format.

Volunteer management also supports the Organizer by maintaining information about individuals who have registered to assist with events. By keeping volunteer records organized, the system can make event-related participation information easier to review and maintain.

10. Participant-Event Assignment Using Edmonds–Karp Maximum Flow

The system shall support participant-event assignment analysis using a network-flow model. This functionality demonstrates how graph algorithms can be applied to allocation problems involving participants, events, and capacity constraints.

The assignment problem can be represented as a directed flow network containing four types of components: a source, participant nodes, event nodes, and a sink. The source is connected to participant nodes, participant nodes are connected to eligible event nodes, and event nodes are connected to the sink.

The capacity of each edge represents the maximum permitted flow through that connection. In a typical model, the source-to-participant edges can represent individual participant availability, while event-to-sink edges represent the available capacity of each event. Participant-to-event edges represent the assignments permitted by the constructed network.

The system uses the Edmonds–Karp algorithm to calculate the maximum flow through this network. Edmonds–Karp is a specific implementation of the Ford–Fulkerson method that uses Breadth-First Search (BFS) to find augmenting paths in the residual graph.

When an augmenting path is found, the algorithm determines the additional flow that can be sent along that path and updates the residual capacities. This process continues until no further augmenting path can be found. The resulting maximum flow represents the maximum assignment supported by the constructed network.

The algorithm has a standard time complexity of $O(VE^2)$, where V is the number of vertices and E is the number of edges.

The system shall construct or process the flow network using the participant-event relationships and capacity information available to the implementation. It shall calculate and display the resulting maximum-flow value for analysis.

This feature demonstrates the use of graph-based optimization in event management. However, the flow calculation is an assignment-analysis operation and should not be interpreted as automatically completing event bookings unless that behaviour is explicitly implemented.

11. Venue Conflict Analysis Using Vertex Cover 2-Approximation

The system shall support the analysis of possible conflicts between events that use the same venue. Venue management is important when multiple events are organized within an institution or organization because the availability of physical spaces may be limited.

For the purpose of conflict analysis, events can be represented as vertices in a graph. An edge between two vertices indicates that the corresponding events share a venue or have been identified as conflicting according to the conflict relationship used by the application. The resulting graph represents the relationships between events that require attention during venue planning.

The system applies the Vertex Cover 2-Approximation algorithm to analyse the conflict graph. A vertex cover is a set of vertices such that every edge in the graph has at least one endpoint included in the selected set. In an event-management context, the selected vertices identify events associated with the conflict relationships represented by the graph.

An approximation algorithm is useful when an optimization problem is computationally expensive to solve exactly. The standard 2-approximation for Vertex Cover repeatedly selects an uncovered edge and includes both of its endpoints in the cover, then removes the edges incident to the selected vertices. The process continues until all edges are covered.

For the standard unweighted Vertex Cover problem, this algorithm produces a cover whose size is at most twice the size of an optimal minimum vertex cover.

The system shall use the constructed conflict graph to calculate an approximate vertex cover and present the result for analysis. This information can help identify events associated with shared-venue conflicts and support further examination by Organizers.

The interpretation of a conflict depends on how the graph is constructed. If the application identifies conflicts only from shared venues, the analysis does not automatically prove that two events overlap in time. Incorporating both venue equality and time overlap would be a useful enhancement.

This functionality demonstrates the application of approximation algorithms to practical event-planning problems and connects graph-based analysis with the management of event resources.

12. Feedback Collection and Management

The system shall allow registered Participants to submit feedback about events in which they have participated. Feedback provides a way to record user opinions and identify areas where future events may be improved.

The feedback functionality supports a rating between 1 and 5 and a text comment. The rating provides a numerical indication of the participant's experience, while the comment allows the participant to describe their observations in greater detail.

Before accepting feedback, the application shall validate the supplied rating and check whether the participant is eligible to submit feedback for the selected event. The current system requirements specify that only registered participants should be considered for event-related participation activities and that duplicate feedback for the same participant and event should be prevented.

The application shall store the accepted feedback in its designated file so that the information can be retrieved later. The stored record should preserve the association between the participant, event, rating, and comment.

Organizers should be able to access feedback-related functionality provided by the application to review the feedback received for events. The information may help identify positive experiences, recurring concerns, and opportunities for improving future activities.

Input validation is important in this module because an invalid rating or repeated feedback entry can reduce the reliability of the stored information. By maintaining structured feedback records, the system supports organized collection and retrieval of participant responses.

13. Attendance Management

The system shall provide attendance-management functionality for recording and viewing participant attendance. Attendance records are important for maintaining information about participation in organized activities.

The Organizer shall be able to record the attendance status of eligible participants. The supported attendance values are Present and Absent. Before processing an attendance operation, the application should verify that the selected event exists and that the participant is associated with the event through an accepted registration.

The application shall validate the attendance status before storing it. Values other than the permitted statuses should be rejected or handled through an appropriate validation message.

Attendance information shall be stored in the designated file so that it can be retrieved later. Participants should be able to view the attendance information available for their registered events, according to the operations supported by the application.

The attendance-management module should maintain a clear association between each record and the corresponding participant and event. This association helps avoid confusion when the system handles multiple events and participants.

The module also supports the overall event-management process by allowing attendance information to be maintained separately from registration and feedback information. Registration indicates that a participant has signed up for an event, whereas attendance records whether the participant was marked Present or Absent.

The system shall ensure that attendance information is recorded consistently and that invalid event identifiers or unsupported attendance values are not processed.

14. File-Based Data Storage and Retrieval

The system shall maintain event-management information using file-based storage. This approach is used in the current Java implementation to preserve information without requiring a separate database management system.

The application maintains separate records for major categories of information, including users, events, participant registrations, volunteer registrations, feedback, and attendance. Separating these categories helps organize the stored information and allows the relevant module to retrieve the records it needs.

When users register, create events, register for activities, submit feedback, or record attendance, the application stores the corresponding information in the designated file. During subsequent operations, the application reads the stored records and uses them to perform the required processing.

Data persistence is necessary because information entered during one operation must remain available after that operation is completed. For example, an event created by an Organizer should remain available when a Participant later searches for events or when the application is restarted and the stored information is loaded again.

The file-handling functionality shall support reading and writing records while handling common file-related errors. If a required file is unavailable or cannot be accessed, the system should display a meaningful message and avoid terminating unexpectedly wherever the implementation permits recovery.

The application should also maintain a consistent record format so that information can be parsed and retrieved correctly. Invalid or incomplete records should be handled carefully to reduce the possibility of incorrect results.

Although file-based storage is suitable for demonstrating persistence in the current implementation, it has limitations when compared with a relational database. A future version could use a database to support structured queries, transactions, constraints, and concurrent access.

15. Input Validation and Duplicate Prevention

The system shall validate important inputs before executing event-management operations. Input validation reduces errors caused by incomplete information, incorrect data formats, invalid identifiers, or values outside the permitted range.

During event creation, the application should validate required event fields, dates, times, and capacity information. During registration, it should verify that the selected event exists, that the registration is not duplicated, and that capacity conditions are satisfied.

During feedback submission, the rating should be restricted to the range from 1 to 5. During attendance management, the status should be limited to Present or Absent. During event-related operations, invalid event identifiers should be detected before records are modified.

Duplicate prevention is particularly important for participant registrations, volunteer registrations, and feedback. Without these checks, the stored files may contain multiple entries representing the same action, resulting in misleading information.

The application should display appropriate messages when inputs are invalid or an operation cannot be completed. These messages help users understand the reason for rejection and provide an opportunity to correct the input.

Validation should be performed at the point where the information is accepted and before the corresponding record is saved. This approach reduces the likelihood of inconsistent data entering the system and improves the reliability of the application.

16. Displaying Operation Results and Error Messages

The system shall communicate the outcome of user operations through clear messages. After a successful operation, the application should inform the user that the requested action has been completed. When an operation fails, the application should explain the problem in understandable terms wherever possible.

For example, after successful registration, the application can display a confirmation message. If a participant attempts to register for an event twice, the system should indicate that the registration already exists. If an event has reached its capacity, the system should communicate that the registration cannot be accepted under the current conditions.

Similarly, when an exact search does not return results, the application should inform the user that no matching event was found. In approximate searching, the system should display suitable candidates according to its implemented matching rules or report that no sufficiently similar result is available.

File-handling operations should also provide suitable feedback when information cannot be read or written. Invalid date formats, unsupported attendance values, incorrect ratings, and invalid event identifiers should be handled through appropriate messages.

This requirement improves the interaction between users and the application. Clear feedback reduces uncertainty, helps users correct mistakes, and makes the menu-driven system easier to operate.

The system should continue operating after recoverable errors rather than terminating immediately. However, if an error prevents a particular operation from being completed safely, the application should stop that operation and avoid saving incorrect information.

2.2 Non-Functional Requirements

Non-functional requirements describe the quality, performance, reliability, usability, and operational characteristics of the **Event Management System (EMS)**. These requirements ensure that the system not only performs the required functions but also operates in a consistent, accurate, maintainable, and user-friendly manner.

A system may provide all the required functions but still be difficult to use if its menus are confusing, unreliable if it loses stored records, or inefficient if it performs unnecessary processing. Therefore, non-functional requirements are essential for evaluating the overall quality of the application.

The Event Management System is developed as a Java-based, menu-driven application with file-based data storage and several algorithmic modules. Its non-functional requirements focus on usability, performance, reliability, data integrity, security, maintainability, scalability, accuracy, data persistence, error handling, portability, and algorithmic efficiency

1. Usability

Usability refers to how easily users can understand, learn, and operate the application. The Event Management System should provide a simple and organized interface that allows users to perform event-related activities without unnecessary complexity.

Since the application supports Organizers, Participants, and Volunteers, the menus should clearly distinguish the functions available to each role. Users should be able to identify the required operation from the displayed options and understand how to proceed after completing an action.

The application should use meaningful menu labels and clear instructions when requesting information. For example, when creating an event, the system should indicate the expected event name, description, date, time, venue, organizer, and capacity. Similarly, registration and feedback operations should explain the information required from the user.

Usability also depends on the quality of the messages displayed by the application. Successful operations should produce confirmation messages, while invalid inputs should result in understandable explanations. Users should not need to interpret technical exceptions or internal algorithmic details to understand ordinary operation results.

The system should maintain a consistent interaction pattern across its menus. After completing an operation, users should be able to return to the relevant menu and select another option without restarting the application unnecessarily.

The application should also present event information in a readable format. Important details, such as event names, dates, times, venues, and capacities, should be displayed clearly so that users can identify the information they need.

A well-organized interface reduces the learning effort required by new users and makes the system more convenient for repeated use. Future versions could improve usability further by introducing graphical interfaces, visual dashboards, search suggestions, and more accessible navigation.

1. Performance

Performance refers to the time and computational resources required by the application to complete its operations. The Event Management System should perform common activities such as viewing events, searching for events, checking registrations, storing records, and retrieving attendance information efficiently.

Searching performance is particularly important because the application demonstrates multiple string-searching algorithms. KMP with LPS is used for exact pattern matching and has a standard time complexity of \(O(n+m)\). This enables the algorithm to avoid repeated comparisons that would otherwise be unnecessary.

Edit Distance using Dynamic Programming supports approximate matching by calculating the minimum editing cost between strings. Its standard implementation requires \(O(mn)\) time and \(O(mn)\) space. The computational cost depends on the lengths of the compared strings, so the application should avoid unnecessary comparisons when they are not required by the search procedure.

Randomized Hashing is integrated into the searching process to filter potential matches. Its effectiveness depends on the hash implementation and input characteristics, and candidate matches must be verified to maintain correctness.

The Edmonds–Karp algorithm is used for participant-event assignment analysis. Its standard worst-case time complexity is \(O(VE^2)\), where \(V\) represents the number of vertices and \(E\) represents the number of edges in the flow network. Vertex Cover 2-Approximation is used for venue-conflict analysis.

Performance also depends on file operations and the volume of stored information. Reading, parsing, and rewriting large files may increase processing time. The system should avoid unnecessary file operations and repeated processing of the same records wherever practical.

The current implementation is intended to demonstrate algorithmic techniques and core event-management operations. Actual performance should be evaluated through testing with representative datasets rather than assumed solely from theoretical complexity.

2. Reliability

Reliability refers to the ability of the system to produce consistent results during repeated operations and to continue functioning correctly under normal operating conditions.

The Event Management System should reliably create events, retrieve stored information, process registrations, manage volunteers, record feedback, and maintain attendance. Repeated execution of a valid operation should produce results consistent with the information stored in the application.

Reliability is especially important when several operations depend on the same event records. For example, an event created by an Organizer must remain available for participant searching and registration. Feedback and attendance records should refer to the appropriate event and participant.

The system should prevent duplicate participant registrations, duplicate volunteer registrations, and duplicate feedback entries for the same participant-event combination. These checks reduce inconsistencies and help ensure that the stored information represents distinct accepted operations.

The application should also handle invalid inputs and recoverable file-related errors without terminating unexpectedly. When an operation cannot be completed, the system should provide a meaningful message and avoid saving a partially processed or invalid record whenever possible.

Reliability is closely connected to the correctness of the algorithms used in the application. KMP should identify exact matches according to its matching rules, Edit Distance should calculate the appropriate editing cost, Maximum Flow should respect the capacities represented in the network, and Vertex Cover should cover the conflict edges in the graph.

The current application uses file-based storage, so reliability also depends on the correct reading, writing, and interpretation of those files. Testing repeated operations, invalid inputs, and application restarts can help identify reliability problems and verify the consistency of stored information.

3. Data Integrity

Data integrity refers to the correctness, consistency, and validity of information maintained by the system. The Event Management System should preserve accurate records of users, events, registrations, volunteering activities, feedback, and attendance.

The application should validate information before accepting it into the stored records. Required event fields should be provided, event identifiers should refer to valid events, ratings should remain within the permitted range, and attendance values should be restricted to Present or Absent.

Event capacity is another important integrity condition. The system should check the available capacity before accepting a new participant registration. Duplicate registration checks should also be performed so that the same participant is not repeatedly registered for the same event through the normal registration process.

Feedback records should maintain a correct relationship between the participant and the event. Similarly, attendance information should be associated with the appropriate registered participant and event. These relationships make it possible to retrieve the correct information when users request their records.

File formats should remain consistent so that records can be interpreted correctly during subsequent executions. If stored records contain missing fields or unexpected values, the application should handle them carefully rather than silently interpreting incorrect information as valid data.

The algorithmic modules must also preserve integrity during calculations. The flow network should use the capacities defined by the assignment model, and the conflict graph should represent the relationships identified by the application. Results should not be presented as accurate if they are based on invalid or incomplete input data.

Maintaining data integrity improves the usefulness of the system because organizers and participants depend on the information displayed by the application to understand event details and participation records.

4. Security

Security refers to the measures used to restrict unauthorized access and protect the information maintained by the application. The Event Management System provides registration, login, and role-based menus to control access to its major operations.

Users should log in before accessing role-specific functionality. The application identifies the user's role and displays the corresponding menu. Organizers, Participants, and Volunteers receive different sets of operations according to their responsibilities.

For example, event creation and attendance management are associated with Organizer functions, while participant registration and feedback submission are associated with Participant functions. Volunteer registration and volunteering-record viewing are associated with Volunteer functions.

The system should avoid exposing operations unrelated to the current user's role through its normal menu workflow. Invalid login credentials should not be accepted, and account-related input should be validated.

However, role-specific menus provide only a basic level of access control. A stronger security design would verify authorization within each sensitive operation instead of relying entirely on menu visibility. It would also use securely hashed passwords rather than storing credentials in plain text.

The current implementation has limitations in password protection and does not include a dedicated security framework. Therefore, it should be described as providing basic login and role-based functionality rather than comprehensive application security.

Future improvements could include password hashing, stronger authentication, session management, authorization checks, secure storage, and audit logging. These enhancements would help protect user information and reduce the possibility of unauthorized access.

5. Maintainability

Maintainability refers to the ease with which the application can be understood, corrected, modified, and extended. The Event Management System should have a logical program structure so that changes to one functionality can be made without unnecessarily affecting unrelated operations.

The Java application includes several distinct functional areas, including user registration and login, event management, searching, participant registration, volunteer management, feedback, attendance, and algorithmic analysis. Organizing these operations into separate methods improves readability and helps developers identify the appropriate location for modifications.

Algorithmic implementations should also remain logically separated from the event-management operations that use them. For example, the KMP implementation can be examined and tested independently from the menu logic that accepts a search query. Similarly, the Maximum Flow and Vertex Cover operations can be maintained separately from the user interface.

File-handling operations should follow consistent conventions so that stored information can be read and written in a predictable manner. Changes to a record format should be considered carefully because multiple modules may depend on the same information.

Meaningful method names, appropriate comments, consistent formatting, and clear validation logic can further improve maintainability. These practices make the code easier to understand during debugging and reduce the effort required to introduce additional functionality.

The modular organization also supports testing. Developers can verify individual algorithms and event-management operations before evaluating the complete application.

Future enhancements, such as a graphical interface, relational database, stronger authentication, or additional search techniques, would be easier to introduce if the system maintains a clear separation between its interface, processing logic, algorithmic operations, and data storage.

6. Scalability

Scalability refers to the ability of a system to accommodate increases in the amount of information it processes or the number of operations it performs.

The Event Management System should support the addition of users, events, registrations, volunteer records, feedback entries, and attendance records within the limits of its current implementation. As more records are added, the application should continue to retrieve and process valid information correctly.

The current implementation uses file-based storage and has implementation-specific limits, including arrays with a maximum capacity of 100 users or events. These limits should be considered when evaluating its scalability. The application should not be described as having unlimited storage or unrestricted support for large numbers of users.

The number of stored records can affect processing time. Event searching may require comparisons against multiple event records, while approximate searching can involve repeated Dynamic Programming calculations. Maximum Flow and conflict analysis also depend on the sizes of their respective graphs.

The file-based approach is suitable for demonstrating persistence and algorithmic operations in a small project. However, managing a large number of records through local files can become difficult because retrieval, updating, and consistency checks may require substantial processing.

A future version could use a relational database to support larger datasets and structured queries. Indexing, pagination, optimized queries, and modular services could further improve performance as the system grows.

Scalability should therefore be understood as a design consideration and a potential area for improvement, rather than a claim that the current application has already been tested with large numbers of concurrent users.

7. Accuracy and Correctness

Accuracy refers to the ability of the application to produce results that correspond to the input information and the rules defined for each operation. Correctness is essential because users depend on the displayed event details, registration status, feedback records, attendance information, and algorithmic results.

The KMP algorithm should identify occurrences of the supplied pattern according to its exact-matching rules. The LPS preprocessing procedure should correctly calculate prefix-suffix information so that the search can reuse previous comparisons.

Edit Distance should calculate the minimum number of permitted editing operations required to transform one string into another. The Dynamic Programming recurrence should correctly consider insertion, deletion, and substitution operations.

Randomized Hashing should be used as a candidate-filtering mechanism. Since hash collisions are possible, candidate matches must be verified before the system treats them as actual matches.

Edmonds–Karp should calculate the maximum flow for the constructed network while respecting edge capacities and updating residual capacities correctly. The final flow value should represent the maximum assignment supported by the network being analysed.

Vertex Cover 2-Approximation should return a set of vertices that covers every edge in the constructed conflict graph. Its approximation guarantee applies to the standard algorithm and graph formulation; it does not guarantee that the selected events represent the best possible venue schedule.

Accuracy also depends on the quality of stored information. If an event has an incorrect date or venue, the algorithmic processing may be correct while the resulting event-management decision is still misleading.

The application should therefore be tested using expected results, sample inputs, boundary cases, and invalid inputs. Verification of both the underlying algorithms and their integration with stored records is necessary to establish the correctness of the complete system.

8. Data Persistence

Data persistence refers to the ability of the application to retain information after an individual operation has finished and make it available during subsequent operations or program executions. The Event Management System uses file-based storage to preserve the records required for its main functions. These records include user information, event details, participant registrations, volunteer registrations, feedback, and attendance.

When a user creates an event, the event information should be written to the relevant file. When a participant registers for an event, the registration should be stored so that it can be retrieved later. Volunteer registrations, feedback entries, and attendance records should likewise remain available after the corresponding operation is completed.

When the application starts, it should load the necessary stored information before allowing the relevant operations to use it. This allows users to work with previously created events and stored participation records instead of having to enter all information again.

Persistence also requires consistent file formats. If the application writes information in a format that cannot be correctly interpreted during the next execution, the records may become unavailable or produce incorrect results. Reading and writing operations must therefore use compatible formats and handle file errors appropriately.

The current system uses local files rather than a relational database. This approach demonstrates the principle of persistent storage but provides fewer facilities for transaction management, concurrent updates, and complex queries.

Future development could introduce a database with suitable constraints and relationships between users, events, registrations, feedback, and attendance. Until then, the current application should be evaluated according to the persistence capabilities provided by its implemented file-handling operations.

9. Error Handling

Error handling refers to the system's ability to detect unexpected conditions, communicate problems clearly, and prevent recoverable errors from disrupting the complete application.

The Event Management System should handle common errors associated with user input, file access, event identifiers, registration requests, date and time formats, ratings, attendance values, and duplicate records.

If a user enters an invalid menu option, the application should display an appropriate message and allow the user to select a valid option. If a date or time does not follow the required format, the application should request valid input rather than storing an incorrect value.

File-related errors may occur when a required file is missing, cannot be read, or cannot be written. The system should report these conditions and handle them without abruptly terminating the application wherever recovery is possible.

The registration process should handle requests for invalid events, repeated registrations, and events whose capacities have been reached. The feedback process should reject invalid ratings and duplicate feedback entries. Attendance management should reject unsupported status values.

Algorithmic modules should also handle appropriate boundary conditions. Examples include empty search patterns, empty event collections, missing candidate matches, and networks that do not contain an augmenting path.

Error handling should not silently convert an unsuccessful operation into a successful one. When a record cannot be stored, the system should not report that the record was successfully saved.

Proper error handling improves usability and reliability by helping users understand what went wrong and by reducing the likelihood that invalid information will be retained.

10. Portability

Portability refers to the ability to run the application in different computing environments with minimal changes, provided the necessary software environment is available.

The Event Management System is implemented in Java. A Java application can generally run on different operating systems that support a compatible Java Runtime Environment, provided that its dependencies and environmental requirements are satisfied.

The application uses file-based storage for its core data-management functions. This avoids requiring a separate database server for the current implementation, although the application still depends on the availability of the required files and appropriate file-system permissions.

Portability also depends on how file locations are defined. If the program assumes a specific absolute path, moving the project to another computer may require changes. Using appropriate relative paths and maintaining a consistent project structure can make deployment easier.

Date and time handling, character encoding, and file access should also be considered when moving the application between environments. Differences in environment configuration can affect how files are located or how information is interpreted.

The system should be tested in the intended Java environment before being used on another machine. Its portability does not automatically mean that it can run without Java or that all operating systems will provide identical behaviour.

Future development could package the application more conveniently and provide clear setup instructions so that other users can install and execute it with less effort.

11. Testability

Testability refers to how easily the application can be checked to determine whether its functions and algorithms produce the expected results.

The Event Management System should support systematic testing of its major operations. These include registration and login, event creation, event viewing, exact searching, approximate searching, participant registration, volunteer registration, feedback collection, attendance management, and file-based data persistence.

Each operation should be tested using valid inputs and invalid inputs. For example, registration testing should include a valid registration, a duplicate registration, an invalid event identifier, and an event whose capacity has been reached.

Algorithmic modules should be tested independently with known examples. KMP can be tested using patterns that occur in the text and patterns that do not occur. Edit Distance can be tested using identical strings, strings with a single character difference, and strings requiring multiple editing operations.

Maximum Flow should be tested with networks having known capacities and expected maximum-flow values. Vertex Cover should be tested using conflict graphs in which the selected cover can be checked against every edge.

File persistence should be verified by storing information, closing the application, restarting it, and checking whether the information can be retrieved correctly.

Testing should also include boundary conditions, such as empty search input, missing records, invalid ratings, and malformed dates. The expected results should be compared with the actual output to identify implementation errors.

A systematic testing process improves confidence in the application and helps establish whether the requirements described in this chapter have been satisfied.

3. SYSTEM DESIGN

The System Design of the Event Management System (EMS) describes the overall structure of the application and the interaction between its major components. The system is designed as a **Java-based, role-oriented application** in which Organizers, Participants, and Volunteers perform different operations. The design also integrates the required Data Structures and Algorithms with event-management operations.

The system follows a layered architectural approach in which each layer is responsible for a particular set of activities. The Users layer represents the individuals interacting with the application. The User Interface Layer accepts user input and displays the results of operations. The Event Management Layer handles the main event-related activities and validates requests. The Algorithm Layer performs the required searching, assignment, and conflict-analysis operations. The Data Storage Layer maintains the records needed by the application and retrieves them when required.

Pattern-matching algorithms provide a foundation for locating a query within a larger text [1].

3.1 High-Level Architecture Diagram

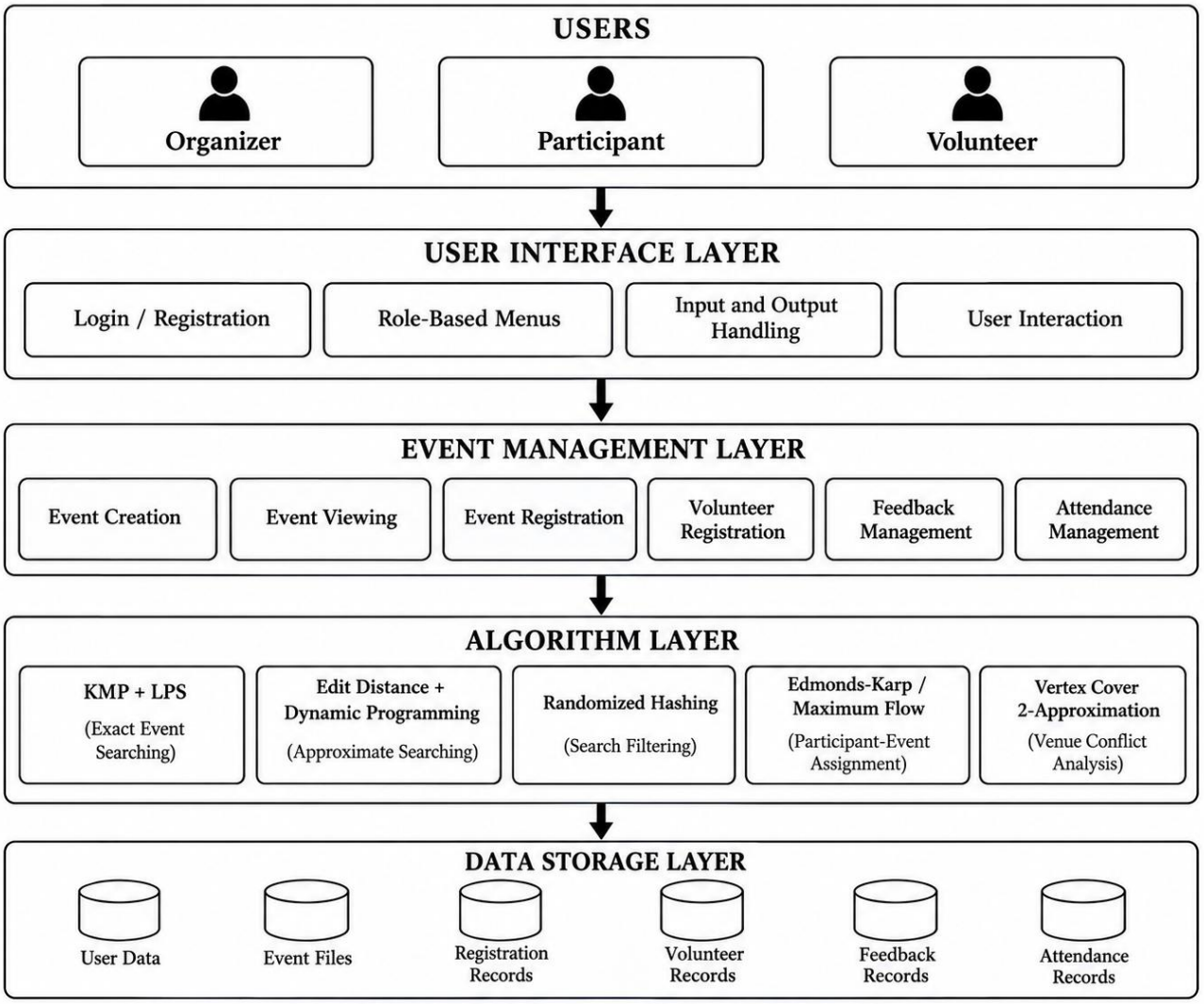


Figure 3.1: High-Level Architecture of the Event Management System

3.2 Description of Each Layer

The High-Level Architecture of the Event Management System consists of five main layers: **Users, User Interface Layer, Event Management Layer, Algorithm Layer, and Data Storage Layer**. Each layer performs a specific function and works together to provide the complete event-management system.

As illustrated in Fig. 3.1, the Users layer initiates the interaction, the User Interface Layer accepts the requests, the Event Management Layer processes event-related operations, the Algorithm Layer performs the required computational tasks, and the Data Storage Layer maintains the records needed by the system.

1. **Users Layer**

The Users layer represents the individuals who interact with the Event Management System. The application supports three major categories of users: Organizers, Participants, and Volunteers. Each category performs different activities according to its responsibilities within the event-management process.

As shown in Fig. 3.1, the Users layer forms the starting point of the system's interaction flow. Users provide the required information, select available operations, and receive the results generated by the application. Their requests are passed through the user interface and processed by the relevant functional components.

Organizer: The Organizer is responsible for creating and managing events. Organizers can enter event details such as the event name, description, date, day, start time, end time, venue, organizer name, and capacity. They can also view available events, perform event searches, examine participant-event assignments, analyse venue conflicts, and manage feedback and attendance through the functions provided by the application.

Participant: Participants use the system to discover events and register for activities that interest them. They can view available events, search using relevant keywords, register for events, view their registrations, submit feedback, and access their attendance information. The system checks registration conditions, including duplicate registrations and event capacity, before accepting a new registration.

Volunteer: Volunteers use the application to explore available events and register for volunteering activities. They can search for suitable events and view their volunteering records. The system maintains volunteer information separately from participant registration records so that the two types of activities can be managed independently.

All three user categories interact with the same application but receive different menus based on their roles. This arrangement organizes the available functions and helps users identify the operations relevant to their responsibilities.

Therefore, the Users layer establishes the connection between the individuals involved in event management and the application's processing components.

2. User Interface Layer

The User Interface Layer provides the interaction between users and the Event Management System. It allows users to enter information, select menu options, submit requests, and view the results produced by the application.

As represented in Fig. 3.1, the User Interface Layer acts as the entry point through which user requests reach the event-management functionality. It accepts the information supplied by the user and directs the selected operation to the corresponding processing method.

The interface provides registration and login facilities. After successful login, the application displays the menu associated with the user's role. This allows Organizers, Participants, and Volunteers to access the functions relevant to their activities without navigating through unrelated options.

For event creation, the interface accepts details such as the event name, description, date, time, venue, organizer, and capacity. For searching, it accepts the search term entered by the user. For registration, feedback, and attendance, it collects the information required by the respective operations.

The interface also displays the results returned by the processing components. For example, it can display event details, search results, registration confirmations, feedback-related information, attendance records, and the results of algorithmic analysis.

Input validation and appropriate messages are important parts of the interaction process. If a user enters an invalid value or selects an unavailable operation, the application should display a suitable message and allow the user to correct the input where possible.

The interface is designed around a menu-driven Java application. Its purpose is to simplify interaction and connect user requests with the appropriate system functionality.

Thus, the User Interface Layer serves as the communication point between the users and the internal components of the Event Management System.

3. Event Management Layer

The Event Management Layer handles the main operational activities of the application. It processes requests received through the user interface and coordinates the functions required to create events, manage participation, maintain volunteer information, collect feedback, and record attendance.

As shown in Fig. 3.1, the Event Management Layer connects user interaction with the application's processing and data-handling operations. It determines which event-management operation must be performed based on the request received from the interface.

Event creation and viewing: This functionality allows Organizers to create events by entering the required information. The system validates the input, prepares the event record, and stores the information for later retrieval. Users can subsequently view the available events and examine their details.

Participant registration: This functionality processes requests from Participants who want to register for events. Before accepting a request, the system checks whether the selected event exists, whether the participant has already registered, and whether the event has sufficient capacity. When the required conditions are satisfied, the registration record is stored.

Volunteer management: This functionality allows Volunteers to register for selected events and view their volunteering activities. It checks for duplicate volunteer registrations and maintains the corresponding records separately from participant registrations.

Feedback management: This functionality supports the collection and retrieval of participant feedback. The application accepts ratings and comments, validates the rating range, and prevents duplicate feedback for the same participant-event combination.

Attendance management: This functionality allows Organizers to record attendance using the supported Present and Absent statuses. The system associates the attendance information with the relevant participant and event and stores the record for later retrieval.

Coordination with algorithmic operations: Certain event-management activities require additional computational processing. Event searching uses the relevant string algorithms, participant-event assignment analysis uses a flow network, and venue-conflict analysis uses a graph-based approximation algorithm. The Event Management Layer coordinates these operations with the relevant algorithmic functionality.

The layer also works with the Data Storage Layer to retrieve existing records and save updated information. This allows event-related activities to use the stored information maintained by the application.

Therefore, the Event Management Layer represents the central operational component of the system. It connects the interface, business operations, algorithmic processing, and stored records to support the complete event-management workflow.

4. Algorithm Layer

The Algorithm Layer contains the Data Structures and Algorithms used to solve specific computational problems in the Event Management System. It supports event searching, approximate matching, participant-event assignment analysis, and venue-conflict analysis.

As illustrated in Fig. 3.1, the Algorithm Layer provides specialized processing for the operations that require algorithmic computation. The algorithms receive the necessary input from the application, process it according to their respective rules, and return results to the relevant event-management operation.

The major algorithms included in the project are described below

KMP with LPS: The Knuth–Morris–Pratt algorithm is used for exact event searching. It preprocesses the search pattern to construct the Longest Proper Prefix which is also a Suffix (LPS) array. The array helps avoid unnecessary comparisons when a mismatch occurs. This enables the system to identify exact occurrences of a search pattern efficiently.

Edit Distance with Dynamic Programming: This algorithm supports approximate event searching. It calculates the minimum number of insertion, deletion, and substitution operations required to transform one string into another. The resulting distance helps identify event information that is similar to the user's search term even when the entered term contains small differences.

Randomized Hashing: Hashing is integrated into the event-searching process as a filtering mechanism. It converts text patterns into numerical hash values and helps identify candidate matches. Since different strings may produce the same hash value, candidate matches must be verified before they are accepted as actual matches.

Edmonds–Karp Maximum Flow: This algorithm is used for participant-event assignment analysis. The problem is represented as a flow network containing a source, participant nodes, event nodes, and a sink. Edge capacities represent the permitted flow through the network, including event-capacity constraints. The algorithm uses Breadth-First Search to identify augmenting paths and calculate the maximum flow supported by the network.

Vertex Cover 2-Approximation: This approximation algorithm is used for venue-conflict analysis. Events are represented as vertices, and edges represent conflict relationships defined by the application. The algorithm selects vertices to cover all edges in the conflict graph and produces an approximate solution to the minimum vertex-cover problem.

These algorithms demonstrate how theoretical concepts can be applied to practical event-management operations. Each algorithm is used for a different purpose, so the system selects the relevant computational technique according to the operation being performed.

The Algorithm Layer also helps separate computational logic from user interaction. This organization makes it easier to examine, test, and maintain individual algorithms without unnecessarily modifying unrelated event-management functions.

Thus, the Algorithm Layer provides the computational foundation for the searching and analysis capabilities of the Event Management System.

5. Data Storage Layer

The Data Storage Layer is responsible for maintaining the information required by the Event Management System. It stores records generated by different operations and retrieves those records when the application needs to display information or perform further processing.

As represented in Fig. 3.1, the Data Storage Layer supports the other components by providing access to persistent information. The current Java implementation uses file-based storage to maintain the records required for event management rather than relying on a separate database system.

User records: The application stores user information required for registration and login. These records allow the system to identify registered users and determine the role associated with an account.

Event records: Event information is stored so that details entered during event creation remain available for subsequent operations. These records contain information such as the event name, description, date, time, venue, organizer, and capacity.

Participant registration records: The system maintains information about participants who have registered for events. These records support duplicate-registration checks and allow participants to view their registered events.

Volunteer records: Volunteer registration information is stored separately so that the system can maintain records of volunteers associated with particular events.

Feedback records: The application stores participant ratings and comments. These records support the retrieval of feedback information for event-related review.

Attendance records: Attendance information is maintained using the supported Present and Absent statuses. The stored records allow attendance information to be retrieved during later operations.

The Data Storage Layer supports persistence by preserving information beyond the completion of an individual operation. When the application starts, the relevant records can be loaded from the files, allowing users to continue working with previously stored information.

The file-handling operations must maintain a consistent record format so that the application can correctly read and interpret the stored information. Appropriate error handling is also required when files are missing, inaccessible, or contain invalid records.

Although file-based storage is suitable for the current project, it has limitations in terms of advanced querying, transaction management, and concurrent access. A future version could introduce a relational database to provide more structured data management.

Therefore, the Data Storage Layer acts as the persistent information component of the Event Management System and supports the operation of its user-management, event-management, searching, registration, feedback, and attendance functions.

4. Implementation

4.1 Module-Wise Implementation

The Event Management System is developed to make event-related activities easier to manage. It provides facilities for handling event information, searching for events, managing participants and volunteers, recording attendance, and collecting feedback. The system is developed using Java, and file handling is used to store and retrieve information.

The implementation is divided into different modules. Each module performs a particular task, which makes the system easier to understand, develop, test, and maintain. Along with the basic event management operations, different algorithms can be used to improve searching, compare text, filter information, allocate limited resources, and identify possible conflicts between events.

The main algorithm-related modules considered in this project are Query-Based Event Searching, Knuth–Morris–Pratt (KMP) String Matching, Edit Distance, Randomized Hashing, Edmonds–Karp Maximum Flow, and Vertex Cover Approximation. Each algorithm solves a different type of problem. Some algorithms work with text, while others work with graphs and resource allocation. The event-search module can use pattern-matching techniques to compare a query with event information [1].

1. Query-Based Event Searching

Introduction

Query-Based Event Searching is a method used to find events based on the information entered by a user. In an Event Management System, many events may be stored with different names, dates, locations, and descriptions. When the number of events increases, checking every event manually becomes difficult and time-consuming.

The searching module allows users to enter a keyword or query to find the events that match their requirements. For example, a user may enter "Cricket" to find cricket-related events. Similarly, users may search using words such as "Workshop", "Music", "Coding", or "Sports Day".

The purpose of this module is to help users find relevant event information without reading every event record. The search process can examine event names and, if supported by the implementation, descriptions or other fields.

Working Principle

The searching process begins when the user enters a search query. The system receives the query and checks it against the available event information. If the query matches an event name or another searchable field, that event is selected as a matching result.

Steps Involved

1. The user enters a keyword or search query.
2. The system reads the available event records.
3. The query is compared with the searchable event information.
4. Matching events are collected.
5. The system displays the matching events.
6. If there are no matching events, an appropriate message is displayed.

Role in the Event Management System

Query-Based Event Searching acts as the entry point for locating relevant event information. It improves the usability of the system by allowing users to search for events according to their interests. More advanced algorithms, such as KMP and Edit Distance, can be used to improve the way the search text is matched. The exact integration depends on the Java implementation.

2. String Matching Using the Knuth–Morris–Pratt (KMP) Algorithm and LPS Table

Introduction

The Knuth–Morris–Pratt algorithm, commonly known as KMP, is a string-matching algorithm used to find a smaller string inside a larger string. The smaller string is called the pattern, while the larger string is called the text.

In an Event Management System, a user may search for an event using a keyword. For example, the user may enter "Cricket" to find an event whose name or description contains that word. String matching provides a systematic way to compare the search pattern with the event text.

A simple searching method may repeat many character comparisons when a mismatch occurs. KMP reduces unnecessary repeated comparisons by using information about the pattern that has already been matched.

The Knuth–Morris–Pratt algorithm uses previously computed prefix information to avoid unnecessary comparisons during string matching [1].

Meaning of the LPS Table

KMP uses a supporting table called the Longest Prefix Suffix table, or LPS table. LPS stands for Longest Proper Prefix which is also a Suffix.

prefix is a group of characters taken from the beginning of a string. A suffix is a group of characters taken from the end of a string. A proper prefix cannot be the complete string itself.

Working Principle of KMP

First, the algorithm receives the text and the search pattern. It creates the LPS table for the pattern. After this, it compares the characters of the pattern with the characters of the text.

When the characters match, both positions move forward. When all the characters of the pattern match, the algorithm reports that the pattern has been found.

If a mismatch occurs after some characters have already matched, KMP does not always restart from the beginning of the pattern. Instead, it checks the LPS value and moves the pattern to a suitable position. The text position does not need to move backwards.

For example, suppose the pattern is "ABAC" and the text is "ABABDABACD". The first few comparisons match "ABA", but the next character does not match the expected character. KMP uses the

LPS information to continue the search efficiently. It eventually finds "ABAC" starting at index 5, where the first character of the text is at index 0.

Steps Involved

1. Read the text and search pattern.
2. Construct the LPS table for the pattern.
3. Compare the pattern with the text character by character.
4. Move forward when characters match.
5. Use the LPS value when a mismatch occurs.
6. Continue until the pattern is found or the text ends.
7. Report the matching position, if one exists.

Role in the Event Management System

KMP can support the event-searching module by finding an exact keyword within event names or descriptions. For example, a search for "Workshop" may locate event descriptions containing that word. It is especially useful when the text is long or the pattern contains repeated characters. It does not automatically find misspelled words; a separate approximate-matching method is needed for that purpose.

3. Advanced Dynamic Programming Using Edit Distance

Introduction

Edit Distance is an algorithmic technique used to measure how different two strings are. It calculates the minimum number of character operations needed to convert one string into another. The common operations are insertion, deletion, and substitution.

Insertion means adding a character. Deletion means removing a character. Substitution means replacing one character with another character.

In an Event Management System, users may enter a search term with a small spelling mistake. For example, the user may search for "Criket" instead of "Cricket". Exact matching may fail to identify the intended event, but Edit Distance can help measure how closely the two words are related.

A smaller edit distance indicates that the two strings are more similar. A larger distance indicates that more changes are required to convert one string into the other. Edit-distance techniques provide a way to measure the difference between two strings [2], [3].

Working Principle

Edit Distance uses a technique called Dynamic Programming. Dynamic Programming solves a large problem by dividing it into smaller problems and storing their answers for later use.

The algorithm creates a table. One string is placed along the rows, and the other string is placed along the columns. Each cell stores the minimum number of operations needed to convert a prefix of one string into a prefix of the other string.

The first row and first column are initialized with values representing the number of insertions or deletions needed when one of the prefixes is empty.

The algorithm then fills the remaining cells one by one. If the current characters are the same, no additional operation is needed, so the value from the previous diagonal cell is used. If the characters differ, the algorithm considers insertion, deletion, and substitution, and chooses the operation with the smallest cost.

The final cell contains the minimum edit distance between the complete strings.

Steps Involved

1. Read the two strings.
2. Create a table with one extra row and column.
3. Initialize the first row and first column.
4. Compare the characters of the two strings.
5. Copy the diagonal value when the characters match.
6. When they differ, calculate insertion, deletion, and substitution costs.
7. Store the minimum value in the current cell.

8. Return the value in the final cell.

Role in the Event Management System

Edit Distance can improve event searching by identifying event names that are similar to the user's query. For example, if a user enters "Criket Tournament", the system can compare the query with "Cricket Tournament" and determine that only a small number of character changes are required.

However, the algorithm does not automatically decide which events should be displayed. The system must apply a rule, such as accepting results below a selected distance threshold. The threshold should depend on the length of the query and the needs of the application.

4. Randomized Hashing and Hash-Based Filtering

Introduction

Hashing is a technique used to convert data, such as a word or string, into a smaller numerical value called a hash value. This value can help compare or organize information efficiently.

In an Event Management System, hashing can be useful when checking keywords or comparing text. Instead of directly comparing every character in every situation, a system may calculate hash values and use them to identify possible matches.

Randomized hashing uses randomly selected parameters or values when calculating hashes. The use of randomness can help reduce certain predictable collision patterns, depending on the hashing method. A collision occurs when two different inputs produce the same hash value.

It is important to distinguish randomized hashing from ordinary hashing. A normal hash function does not necessarily use randomness. If the Java program only calculates standard hash values, the report should describe it as hash-based filtering rather than claiming that randomized hashing has been implemented. Randomized pattern-matching methods can use compact fingerprints to compare strings efficiently [4].

Working Principle

The hashing process begins by taking a string as input. The characters are converted into a numerical representation, and a hash function combines them to calculate a hash value.

In a randomized hashing method, the algorithm may choose a parameter randomly from an allowed range and use it in the hash calculation. The exact process depends on the chosen algorithm.

Steps Involved

1. Read the search query.
2. Select the hash function and any required parameters.

3. Calculate the hash value of the query.
4. Calculate or retrieve hash values for the searchable data.
5. Compare the hash values.
6. Perform direct string comparison when necessary.
7. Display only the verified matching results.

Role in the Event Management System

Hash-based filtering can support the searching module by identifying event records that are possible matches for a query. It can be particularly useful when the number of records grows. Nevertheless, hashing should not be treated as proof that two strings are equal. The actual strings must be checked whenever correctness requires an exact match.

5. Network Flow Using the Edmonds–Karp Algorithm

Introduction

The Edmonds–Karp algorithm is used to find the maximum flow in a flow network. A flow network is a directed graph in which each connection has a capacity. Capacity represents the maximum amount of flow that can pass through a connection.

The algorithm is based on the Ford–Fulkerson method and uses Breadth-First Search (BFS) to find augmenting paths. An augmenting path is a route from a source node to a destination node through which additional flow can be sent.

In an Event Management System, a flow network can be used to model certain resource-allocation problems. For example, participants may be connected to events they are eligible or willing to attend, while event nodes have limits on the number of available places.

The algorithm can calculate the maximum number of assignments possible under the capacity constraints represented in the network. The network must be designed correctly to reflect the real rules of the application. Network-flow algorithms provide methods for finding feasible flows through a graph subject to capacity constraints [5].

Important Terms

- Source: The starting node from which flow enters the network.
- Sink: The final node where the flow is collected.
- Capacity: The maximum flow allowed through an edge.
- Residual capacity: The additional flow that can still be sent through an edge.

- Augmenting path: A path from source to sink with available residual capacity.
- Maximum flow: The greatest total flow that can be sent from the source to the sink without exceeding the edge capacities.

Working Principle

First, the system represents the problem as a graph. The graph contains a source, participant nodes, event nodes, and a sink.For example, the source can connect to participant nodes with capacity 1, meaning that each participant can receive at most one assignment in the model. Participant nodes connect to the events they can attend. Event nodes connect to the sink with capacities equal to the available places for those events.The algorithm begins with zero flow. It then performs BFS to find a path from the source to the sink using edges that still have available residual capacity.After finding a path, the algorithm identifies the smallest residual capacity along that path. This value is the maximum additional flow that can be sent through the path. The algorithm increases the flow by this amount and updates the residual capacities.The algorithm repeats the process until BFS cannot find another augmenting path. At that point, no additional flow can be sent under the network's capacity constraints, and the current flow is the maximum flow.

Steps Involved

1. Represent participants, events, and assignment rules as a flow network.
2. Set the capacity of every edge.
3. Initialize all flows to zero.
4. Use BFS to find an augmenting path from the source to the sink.
5. Find the minimum residual capacity along the path.
6. Increase the flow along the path by that amount.
7. Update the residual network.
8. Repeat until no augmenting path exists.
9. Return the total maximum flow.

Role in the Event Management System

Edmonds–Karp can be used to model participant allocation when the problem can be represented as a flow network. It helps calculate how many assignments are possible while respecting event capacities and eligibility constraints.

However, maximum flow does not automatically solve every scheduling problem. If the application needs to consider time conflicts, participant preferences, or multiple simultaneous assignments, those rules must be represented in the network or handled by additional logic. The algorithm should be described as an implemented module only if the Java project actually builds and processes such a network.

6.NP-Completeness and Approximation Using Vertex Cover

Introduction

Vertex Cover is a graph problem used to select a set of vertices so that every edge in the graph has at least one selected endpoint. A graph consists of vertices, which represent objects, and edges, which represent relationships between those objects.

Finding the smallest possible vertex cover is an NP-hard optimization problem. This means that no polynomial-time algorithm is currently known for finding the exact minimum solution for every possible input. For large graphs, an approximation algorithm can be used to obtain a solution efficiently, even if it is not always the smallest possible solution.

In an Event Management System, a graph can represent possible conflicts between events or venues. For example, two events may be connected by an edge if they require the same venue at overlapping times. Selecting vertices to cover all conflict edges can help identify a set of events that should be reviewed for rescheduling.

The meaning of the selected vertices depends on how the graph is designed. A vertex cover identifies a set of vertices that touches every edge; it does not, by itself, determine which event should be cancelled or rescheduled. Vertex-cover problems are studied in computational complexity, and approximation methods can provide practical solutions for large instances [6], [7].

Working Principle

First, the system creates a graph. Each vertex represents an event or another relevant item, and an edge connects two vertices when a conflict exists between them.

The objective is to select vertices so that every conflict edge has at least one selected endpoint. A minimum vertex cover contains the smallest possible number of vertices that satisfy this condition.

Finding an exact minimum vertex cover can be computationally difficult for large graphs. Therefore, a common approach is a 2-approximation algorithm.

The algorithm begins with an empty cover. It selects an uncovered edge and adds both endpoints of that edge to the cover. All edges connected to either endpoint are then considered covered. The process continues until no uncovered edges remain.

This approach is easy to implement and runs efficiently, but it may select more vertices than the true minimum. Its standard guarantee is that the cover it returns contains at most twice as many vertices as an optimal minimum vertex cover.

Steps Involved

1. Represent events or other items as graph vertices.
2. Create an edge for every conflict that must be covered.
3. Initialize the vertex cover as an empty set.
4. Select an uncovered edge.
5. Add both endpoints of that edge to the cover.

6. Mark every edge touching either endpoint as covered.
7. Repeat until all edges are covered.
8. Return the selected set of vertices.

Role in the Event Management System

Vertex Cover Approximation can support venue-conflict analysis by identifying a set of events that collectively touch every conflict represented in the graph. The organizer can then examine these events and decide which ones need to be moved, rescheduled, or reviewed. The algorithm does not automatically determine the best new venue or time. Those decisions require additional scheduling rules and information. Its role is to provide a manageable set of conflict-related events for further examination.

4.2 USER INTERFACE AND MENU LOGIC

The user interface is the part of the Event Management System through which users interact with the application. Since the project uses a Java console interface, users interact with the system by entering menu options and providing information through the keyboard. The menu is designed to present the available operations clearly. Depending on the permissions and role of the user, the system may provide options for creating events, viewing event information, registering participants, managing volunteers, recording attendance, and collecting feedback. When the program starts, it displays the appropriate initial menu. The user selects an operation, and the program executes the corresponding logic. After completing the operation, the system can return to the menu so that the user can continue using the application. The menu-driven structure separates the different operations and makes the program easier to use. It also helps organize the Java code because each menu option can call a separate method. Input validation is an important part of the interface. If a user enters an invalid menu number or provides information in an incorrect format, the system should display an understandable message rather than terminate unexpectedly.The interface also communicates the results of operations. For example, it can display whether an event was created successfully, whether a participant was registered, or whether an entered search query returned any matching results.

4.3 EVENT MANAGEMENT

The Event Management module handles the creation, storage, display, and updating of event information, depending on the features implemented in the program. It provides a central place for managing the events that participants may want to attend. Each event may contain details such as the event name, date, time, venue, description, and organizer information. These details help users understand when and where

an event will take place. When an authorized user creates an event, the program receives the necessary information, checks the input, and stores the event record. The system can later display the event information when requested. Updating event details is also important because event schedules and venues may change. If the application supports modification or deletion, it should ensure that the corresponding stored record is updated correctly. The module can work with the searching algorithms described earlier. Query-based searching and string matching can help users find relevant events, while Edit Distance can be used to compare similar queries. These algorithms should be connected to the event records through the appropriate Java methods.The event management module therefore acts as a central part of the application. It maintains the information required by registration, attendance, feedback, and other event-related operations.

4.4 PARTICIPANT AND VOLUNTEER MANAGEMENT

Participant Management handles the information associated with users who register for events. The system may record participant details and the events for which they have registered. These records help the organizer understand who is expected to attend an event. When a participant registers, the system receives the required details and checks the relevant input. If the registration is valid, the program stores the registration information using the storage mechanism implemented in the project. Volunteer Management focuses on maintaining volunteer information and relating volunteers to event activities when this functionality is supported. Volunteers may assist with registration, guiding participants, managing event activities, or helping organizers perform event-related tasks.The system should keep participant and volunteer records organized so that they can be retrieved when required. For example, an organizer may need to view registrations for an event or check which volunteers are associated with it.If the project includes participant allocation subject to event capacities, a network-flow model can be used to study how participants may be assigned to available places. Such a model must include the correct eligibility and capacity rules. A basic registration feature does not automatically mean that the Edmonds–Karp algorithm is implemented. Separating participant and volunteer management from event management improves the organization of the code. It also makes the application easier to maintain because changes to one type of record do not require every part of the system to be rewritten.

4.5 FEEDBACK AND ATTENDANCE MANAGEMENT

Feedback and Attendance Management supports the collection of information about event participation and user experience. Attendance records help the organizer determine whether registered participants attended an event. Depending on the program's design, attendance may be stored against a participant, an event, or a registration record. When attendance is recorded, the system should identify the relevant participant and event and store the attendance status. Later, the organizer can retrieve this information to review participation. Feedback allows participants to share their opinions about an event. Feedback may include comments or a rating, depending on the fields provided by the application. The system stores the feedback so that it can be viewed later and used to understand participant experiences. For example, participants may provide comments about the event organization, venue, activities, or schedule. Such information can help organizers identify areas that may need improvement. File handling provides persistence for attendance and feedback records. The program must read the stored information when displaying existing records and save changes when new attendance or feedback is entered. The module helps the Event Management System maintain information beyond event creation and registration. It supports the review of completed events and provides useful records for future planning.

4.6 ALGORITHM INTEGRATION AND COMPLEXITY ANALYSIS

The algorithm modules described in this chapter address different types of problems. Query-Based Event Searching locates relevant records, KMP searches for exact character patterns, and Edit Distance measures the number of changes required to convert one string into another. Hash-based filtering can help identify possible matches. Edmonds–Karp calculates maximum flow in a capacity-based network, while Vertex Cover Approximation identifies a set of vertices that covers every edge in a graph. These algorithms are not interchangeable. Each one has a particular purpose, and the system must provide suitable input for the algorithm to produce meaningful results. For example, text-searching algorithms require strings such as event names, descriptions, or search queries. A maximum-flow algorithm requires a graph with capacities. Vertex Cover requires a graph in which edges represent relationships or conflicts. The application must convert its event information into the required input format before these graph algorithms can be used. Here, N and M represent the lengths of strings or patterns where applicable, L represents the amount of event text examined per record, R represents the number of matching results, V represents the number of graph vertices, and E represents the number of graph edges. The table gives typical or standard complexity bounds under the assumptions stated for each algorithm. Actual performance depends on the data structures and implementation used in the Java program.

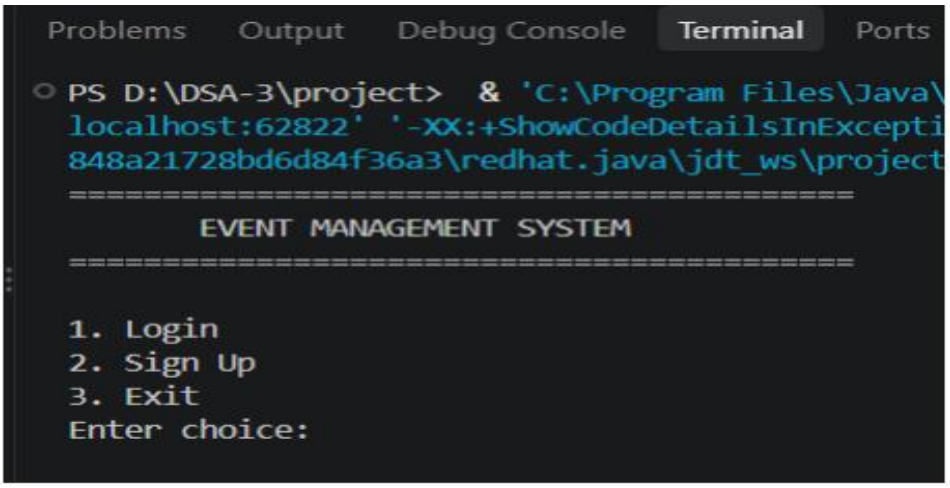
Integration with Event Management Operations

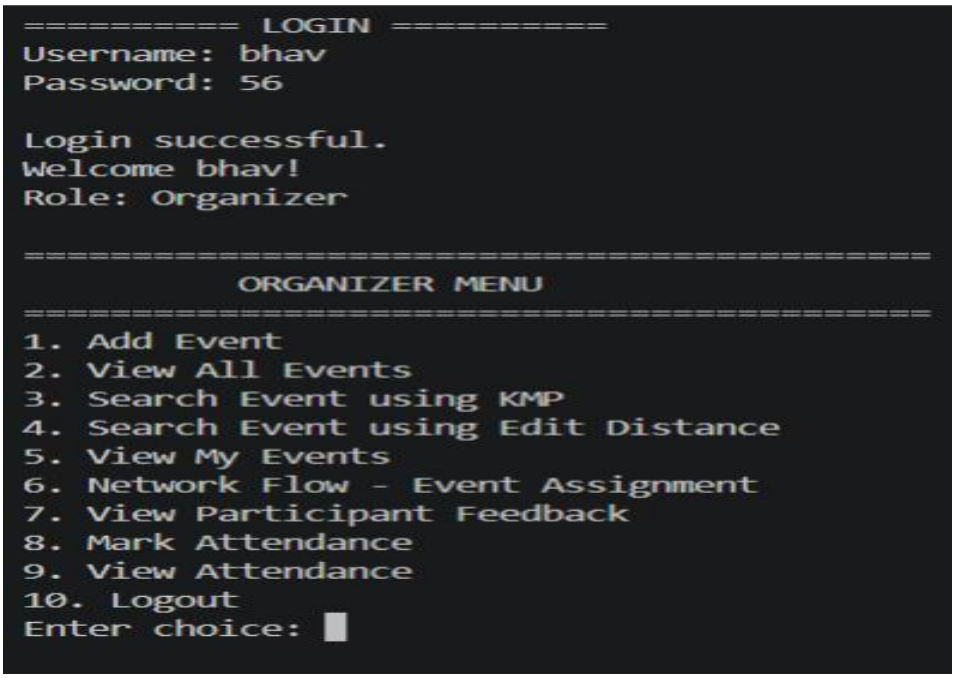
The searching algorithms can support the process of finding events by name or keyword. KMP is useful when exact character matching is required, while Edit Distance can measure how closely a query resembles an event name. Hash-based filtering may reduce unnecessary comparisons when implemented appropriately.

The graph algorithms address different planning problems. Edmonds–Karp can calculate the maximum number of assignments in a correctly designed capacity network. Vertex Cover Approximation can identify a set of vertices associated with every conflict edge in a graph.These algorithms may be integrated with the existing event, registration, attendance, and feedback modules through Java methods. Their outputs can be displayed through the console interface and stored when the application requires persistent records.The separation of algorithm logic from file handling and menu logic makes the program easier to maintain. For example, a searching method can receive event text as input and return matching results without being responsible for displaying the entire menu. Similarly, a graph algorithm can process a graph and return a result that another method interprets.

5. RESULTS

5.1 Result Screenshot





```
========== ADD EVENT ==========
Enter event name: drawing
Enter event description: creativity
Enter date (DD-MM-YYYY): 20-10-2026
Enter start time (HH:MM): 02:00
Enter end time (HH:MM): 04:00
Enter venue: H401
Enter capacity: 20

Event created successfully.
Event ID: 21
Event file: event21.txt
Day: TUESDAY
```

Problems Output Debug Console **Terminal** Ports

```
PS D:\DSA-3\project>  & 'C:\Program Files\Java\jdk-21.0.10\bin\java.exe' '-agentlib:jdwp=transport=
localhost:62822' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ADMIN\AppData\Roaming\Co
848a21728bd6d84f36a3\redhat.java\jdt_ws\project_e01748e9\bin' 'EventManagementSystem'
Time        : 10:00 - 15:00
Venue       : Project Hall
Organizer   : Bhavana
Capacity    : 100

----------------------------------------
Event ID    : 20
File        : event20.txt
Name        : Annual Coding Festival
Description: A technical festival featuring coding challenges workshops and project demonstrations
Date        : 23-11-2026
Day         : MONDAY
Time        : 09:00 - 18:00
Venue       : University Grounds
Organizer   : Bhavana
Capacity    : 200

----------------------------------------
Event ID    : 21
File        : event21.txt
Name        : drawing
Description: creativity
Date        : 20-10-2026
Day         : TUESDAY
Time        : 02:00 - 04:00
Venue       : H401
Organizer   : bhav
Capacity    : 20
```

```
========== KMP EVENT SEARCH ==========
Enter search pattern: java

Event ID: 2
Name: Web Development Seminar
Date: 28-09-2026
Venue: Seminar Hall

Algorithm: KMP
Module: 2 - String Algorithms
Time Complexity: O(n + m)
```

```
========== EDIT DISTANCE SEARCH ==========
Enter event keyword: ddrawing

Event ID: 21
Name: drawing
Edit Distance: 1
Date: 20-10-2026
Venue: H401

Algorithm: Edit Distance
Module: 3 - Advanced Dynamic Programming
Time Complexity: O(n*m)
```

```
========== MY EVENTS ==========
21. drawing | 20-10-2026 | H401
```

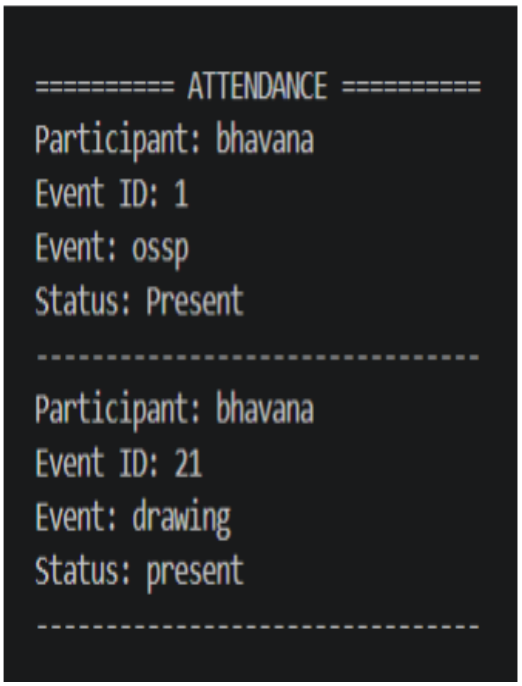
```
========== NETWORK FLOW ==========
Algorithm: Edmonds-Karp
Network:
Source -> Participants -> Events -> Sink

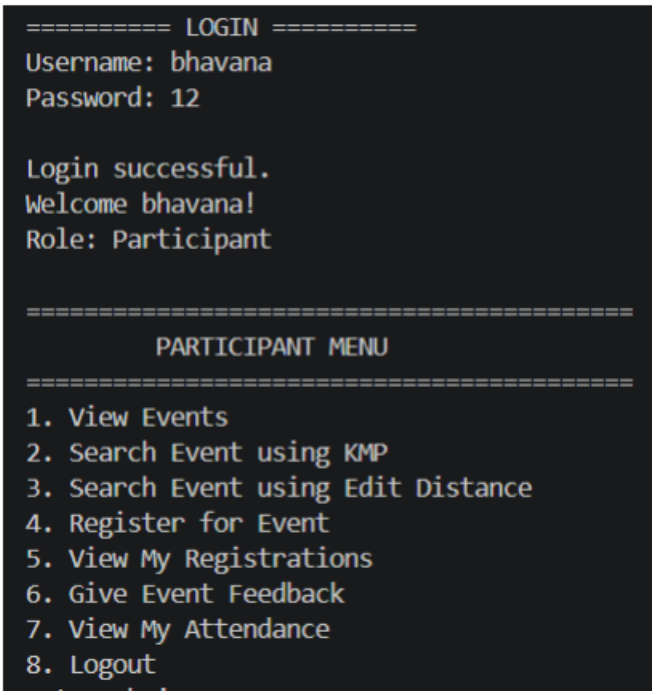
Maximum Flow = 2
Maximum participant-event assignments calculated.
Venue conflict events analyzed: 10
```

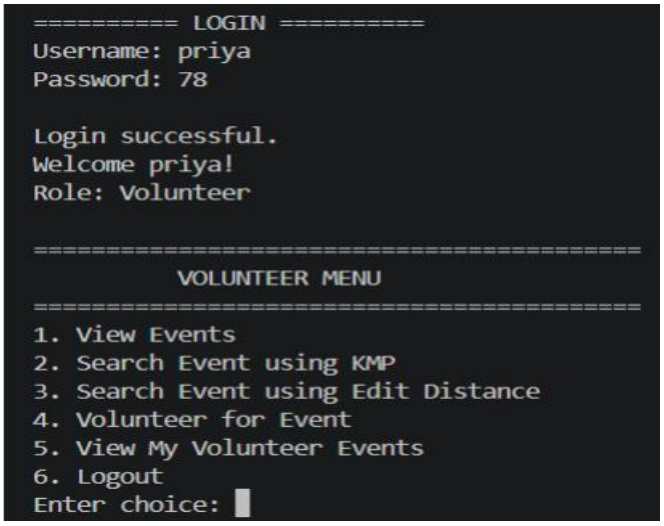
```
========== PARTICIPANT FEEDBACK ==========
Participant: bhavana
Event ID: 1
Event: os
Rating: 4/5
Feedback: good
--------------------------------
Participant: bharath
Event ID: 4
Event: Hackathon 2026
Rating: 4/5
Feedback: good
--------------------------------
```

```
----------------------------------------
Enter Event ID: 21

Registered Participants:
- bhavana
Enter participant username: bhavana
Enter status (Present/Absent): present
Attendance saved.
```







5.2 Functional Testing

Test Area	Expected Result
Sign Up	Account is stored with selected role; duplicate usernames rejected.
Login	Valid credentials open the correct role menu.

Add Event	Valid event details are saved with an event ID.
Exact Search	Matching events are returned using KMP.
Approximate Search	Events with minimum edit distance ≤ 2 are returned.
Registration	Registration succeeds when not duplicated and capacity is available.
Volunteer	Volunteer record is stored without duplication.
Feedback	Registered participant can submit one rating/comment.
Attendance	Organizer can mark Present/Absent.
Network Flow	Maximum participant-event flow is calculated.
Conflict Analysis	Shared-venue conflicts are analyzed with vertex cover.

6. CHALLENGES AND LIMITATIONS

The development of an Event Management System involves several challenges related to program design, data storage, user input, event searching, participant management, and the correct use of algorithms. The main purpose of the project is to simplify event-related activities by organizing information and providing different operations through a Java-based application. Although the system helps manage event information more systematically, developing and maintaining it requires careful planning and testing.

During development, it is important to ensure that event details are stored correctly, user inputs are validated, and the different modules work together without creating inconsistencies. For example, when an event is created, its information must be stored properly so that it can be displayed or searched for later. Similarly, participant registrations, volunteer details, attendance records, and feedback information must be handled carefully to avoid incorrect or incomplete records.

Another challenge is selecting appropriate algorithms for different operations. Searching for an event, comparing two text strings, calculating possible participant assignments, and identifying conflicts are different problems that require different approaches. An algorithm that performs well for one task may not be suitable for another. Therefore, the purpose, working procedure, time complexity, and memory requirements of each algorithm must be understood before it is integrated into the system.

The current project also has limitations because it uses a Java console interface and file-based storage. These choices make the project easier to understand and demonstrate in an academic environment, but they restrict some advanced features that may be available in larger event management applications. Approximate matching techniques can help address spelling differences, although they may require additional computation [3], [8].

6.1 CHALLENGES FACED DURING DEVELOPMENT

1. **Designing a Modular System**

One of the main challenges in developing the Event Management System is organizing the program into separate modules. An event management application contains several related operations, including event creation, event searching, participant registration, volunteer management, attendance recording, and feedback collection. If all these operations are written as one large block of code, the program becomes difficult to understand, test, and modify. For example, the logic for creating an event is different from the logic for registering a participant. Event creation requires information about the event, such as its name, date, time, and venue. Registration requires information about the participant and the event they want to attend. Although these operations are related, each has a different purpose and should be handled separately.

To address this challenge, the system can be organized into methods or modules, with each module responsible for a particular task. The menu logic handles user choices, event-related methods handle event records, and searching methods process queries. File-handling methods are responsible for reading and writing stored information.

A modular structure makes the code easier to maintain because changes can be made to one part without unnecessarily affecting the entire application. It also makes testing easier, as individual methods can be checked separately before testing the complete system.

2. Managing Event and User Information.

Another important challenge is maintaining different types of information in an organized manner. The system may need to store event details, participant records, volunteer information, registrations, attendance, and feedback. Each type of information has its own fields and relationships with other records.

For example, a participant may register for a particular event, and several participants may register for the same event. The system must preserve the relationship between the participant and the selected event. If the records are stored incorrectly, it may become difficult to identify which participants belong to a particular event.

The challenge becomes more significant when the same information is required by multiple modules. The event list may be needed for searching, registration, and attendance operations. Therefore, the system must retrieve and process the stored information consistently.

This challenge can be reduced by defining a clear structure for each type of record and using consistent methods to create, retrieve, update, and display information. The program should also validate important fields before accepting a record. For example, an event name should not be accepted as empty if it is required for identifying the event.

3. File Handling and Data Persistence

File handling is an important part of the project because the system uses files to retain information between program executions. Unlike data stored only in variables, file-based records can be retrieved after the program is closed, provided that the data has been saved successfully.

However, working with files introduces several challenges. The program must identify the correct file, use a consistent format, handle missing files, and manage reading and writing operations carefully. Incorrect file operations may cause information to be lost, duplicated, or stored in an unexpected format.

For example, suppose participant registrations are stored in a text file. When the program starts, it must read the existing registrations correctly. When a new registration is added, the system must save it without accidentally removing the previous records. If the file is rewritten incorrectly, earlier registration details may be lost.

Another difficulty occurs when the stored data does not follow the expected format. A missing field, an unexpected separator, or an incomplete record may cause problems when the program tries to process the information.

These challenges can be reduced by using consistent record formats, carefully managing file operations, checking for input and output errors, and testing the program with both existing and newly created files. The program should also provide understandable messages when a file cannot be accessed.

4. Handling Invalid User Input

The console-based interface accepts information directly from users. Users may enter invalid menu choices, leave required fields empty, enter dates in the wrong format, or provide values that do not match the expected data type.

For example, the system may request a date in a specific format, but the user may enter the month and day in the wrong order. Similarly, a menu may expect a number, but the user may enter letters. If such inputs are not handled properly, the program may display incorrect results or terminate unexpectedly.

Input validation is therefore necessary to maintain the reliability of the application. The system should check whether the entered values follow the expected format before processing them. Where possible, it should display an error message and allow the user to enter the information again.

For date-related operations, Java date-handling classes can help verify whether a value follows the required format. Menu inputs can also be checked to ensure that the selected option is available.

Proper input validation improves the user experience and reduces errors in the stored records. However, validation must be applied consistently across all modules to be effective.

5. Implementing Efficient Event Searching

Searching becomes more challenging as the number of stored events increases. When only a few events are available, a simple search may be sufficient. However, when many events contain different names and descriptions, checking each record can require more processing.

Another difficulty is deciding what should count as a match. An exact search returns a result only when the required text matches according to the search rules. A user who enters a misspelled event name may not receive the intended result even if the correct event exists.

To address these issues, string-searching techniques can be considered. The Knuth–Morris–Pratt algorithm uses an LPS table to avoid unnecessary repeated comparisons when searching for a pattern. Edit Distance measures how many character operations are required to convert one string into another, making it useful for identifying similar words.

Hash-based filtering can also be used to identify possible matches, depending on the implementation. However, matching hash values alone cannot prove that two strings are equal because collisions are possible.

The challenge is not simply selecting an algorithm; the system must also connect the algorithm correctly to the event data and apply suitable matching rules. Search results should be checked carefully to ensure that relevant events are displayed and unrelated records are not incorrectly treated as matches.

6 .Understanding and Applying Algorithm Complexity

Another challenge is understanding the time and space requirements of the algorithms used in the project. Each algorithm solves a different problem and has different computational requirements. Selecting an algorithm without understanding its complexity may result in unnecessary processing or excessive memory usage.

For example, KMP performs exact pattern matching in O(N + M) time, where N is the text length and M is the pattern length. Edit Distance generally requires O(M × N) time and O(M × N) space when a complete dynamic programming table is stored. This difference is important because Edit Distance may require more computation when both strings are long.

Graph algorithms introduce additional complexity. Edmonds–Karp has a standard worst-case time complexity of $O(V \times E^2)$, where V is the number of vertices and E is the number of edges. Vertex Cover Approximation can be implemented efficiently using suitable graph structures, but its result may not be the smallest possible cover.

The challenge is to understand both the theoretical behaviour of the algorithms and their practical suitability for the project. This can be addressed by studying the algorithm step by step, testing it with small examples, checking the output, and measuring its behaviour on larger inputs where appropriate.

7.Designing Graph-Based Resource Allocation

Graph-based algorithms are more difficult to understand and implement than straightforward searching operations because they require the problem to be represented correctly as a graph.

For maximum-flow calculations, the system must define the source, sink, vertices, edges, and capacity of each edge. If participant allocation is being modelled, the graph must also represent the relevant event capacities and participant eligibility rules.

An incorrectly designed network may produce a mathematically correct answer to the wrong problem. For example, if the network does not represent an event's actual capacity, the maximum-flow result may not reflect the number of participants that can really be accommodated.

Vertex Cover also requires a suitable graph. Its vertices and edges must represent the relationships or conflicts that the system wants to examine. A conflict between two events can be represented as an edge only when the project defines that relationship clearly.

This challenge can be addressed by first describing the real-world problem, then designing a small graph, assigning capacities or conflict edges, and testing the algorithm with simple examples. The results should be checked manually before using larger inputs.

It is also important to understand that these algorithms solve specific mathematical problems. Maximum flow does not automatically handle every scheduling restriction, and Vertex Cover does not independently decide the best venue or time for an event.

8.Integrating Different Modules

The different modules of the Event Management System depend on one another. For example, participant registration depends on event information, attendance depends on registration or participant records, and searching depends on the availability of event data.

If modules use inconsistent record formats or different assumptions about the same information, the application may produce unexpected results. An event may be displayed in one part of the program but not found by another operation if the data is loaded or processed differently.

Integration also becomes challenging when algorithmic methods are connected to the existing menu and file-handling logic. The program must pass the correct input to each method and process the returned result properly.

A useful approach is to test each module separately and then test the complete workflow. For example, the system can be tested by creating an event, saving it, retrieving it, searching for it, registering a participant, and checking the related records. This helps identify problems that may not appear when a method is tested alone.

6.2 LIMITATIONS OF THE CURRENT SYSTEM

1. Console-Based User Interface

The current project uses a Java console interface rather than a graphical web or mobile interface. Users interact with the application by reading menu options and entering values through the keyboard.

This approach is suitable for demonstrating program logic and algorithm implementation. It also allows the developer to focus on the core operations without first building a complex graphical interface.

However, a console interface has limited visual presentation. It cannot provide the same type of interactive forms, graphical calendars, event cards, dashboards, or visual schedules that are common in modern event management applications.

Users may also need to enter several values manually, which can make some operations less convenient. Displaying a large amount of event information in a console may be less readable than presenting it in a structured table or graphical interface.

A graphical interface could improve usability in the future, but the absence of such an interface does not prevent the current console application from demonstrating its basic event management functionality.

2. File-Based Storage

The project uses file-based storage to maintain information. This approach is straightforward and suitable for a small academic application because it avoids the additional setup required for a database server.

However, files provide limited support for complex queries and relationships between records. As the number of events, registrations, volunteers, and feedback entries grows, reading and rewriting files may become less efficient.

Maintaining consistency across several files can also be difficult. For example, an event may have related participant registrations and attendance records. If one file is updated while another is not, the records may become inconsistent.

File-based storage also does not automatically provide database features such as transaction management, concurrent updates, and relational constraints. These limitations become more important when multiple users need to update related records at the same time.

A relational database could provide more structured storage, efficient queries, and stronger consistency controls in a future version. However, the current implementation should be described according to the storage mechanism actually present in the source code.

3. Limited Support for Simultaneous Users

A console application that stores data in local files is not designed automatically to support many users modifying the same information simultaneously.

For example, if two users attempt to register participants or update an event at nearly the same time, their operations may interfere with one another if the program does not include suitable synchronization and data-management mechanisms.

The problem becomes more significant when the system is expected to support users from different computers. A local file on one computer is not automatically available to users on other computers, and the application does not automatically coordinate updates across separate copies of the program.

The current project can demonstrate the main event management operations, but reliable multi-user access would require additional architecture and concurrency controls. A shared database and a suitable backend service could support such requirements in a future version.

4. Limited Search Capabilities

The effectiveness of event searching depends on the search rules implemented in the application. A basic exact search may fail when the user enters a misspelled word, uses an unexpected abbreviation, or searches with a related term rather than the actual event name.

Edit Distance can help compare similar strings, but it does not automatically understand the meaning of a query. For example, it cannot determine by itself that two completely different words refer to the same sport or activity.

KMP is designed for exact pattern matching, so it does not directly solve the problem of semantic similarity or spelling correction. Hashing also cannot determine whether two different words have similar meanings.

More advanced search facilities could include filters for dates, categories, venues, and event types, as well as spelling suggestions or ranking of results. Such features require additional rules and data structures and should be treated as extensions unless they already exist in the current code.

5. Limitations of Dynamic Programming for Long Strings

Edit Distance provides a useful way to compare two strings, but its standard dynamic programming implementation has a limitation: it requires $O(M \times N)$ time and $O(M \times N)$ space when a complete table is stored.

When both strings are short, this cost is usually manageable. However, when the strings become much longer, the number of table cells increases substantially. This can lead to increased processing time and memory usage.

The algorithm also measures the number of insertions, deletions, and substitutions rather than the meaning of the text. Two words with a small edit distance are not necessarily related in meaning, and two words with a large edit distance may still refer to related concepts.

Therefore, Edit Distance should be used as a text-similarity measure rather than a complete language-understanding solution. A future version could use memory optimization or additional search rules to improve efficiency and relevance.

6. Hash Collisions and Hashing Limitations

Hashing can help compare or filter data, but a hash value is not a unique representation of every possible string. Different strings can produce the same hash value, which is known as a collision.

If a system treats matching hash values as proof that two strings are equal, it may produce incorrect results. A reliable implementation must verify the actual strings when exact matching is required.

The effectiveness of randomized hashing also depends on the hash function and the way its parameters are selected. Simply calculating a normal hash value does not mean that the program uses randomized hashing.

Another limitation is that hashing does not directly measure how similar two strings are. For example, two words that differ by one character may have very different hash values. Therefore, hashing alone is not a replacement for Edit Distance or a string-matching algorithm.

7. Limitations of Maximum-Flow-Based Allocation

Edmonds–Karp can calculate maximum flow in a network, but its output is only as useful as the network model provided to it.

If the system represents participant-to-event assignments using a flow network, it must correctly define participant eligibility, event capacity, and the permitted assignments. Missing or incorrect constraints can lead to an allocation that is mathematically valid but unsuitable for the real event.

Maximum flow also does not automatically optimize every possible preference. For example, participants may prefer certain event times, venues, or activities. A standard maximum-flow model does not automatically select the assignment that best satisfies all such preferences unless the problem is extended with suitable constraints or a different optimization method.

The standard worst-case time complexity of Edmonds–Karp is $O(V \times E^2)$. This may become expensive for very large networks containing many participants, events, and connections.

Consequently, the algorithm is useful for specific capacity-based allocation problems, but it does not replace a complete scheduling system. More advanced requirements would need additional modelling and optimization.

8. Limitations of Vertex Cover Approximation

The Vertex Cover 2-approximation algorithm provides a valid cover with a known approximation guarantee, but it does not always return the smallest possible cover.

For example, in a graph containing a path of four vertices, the algorithm may select both endpoints of multiple edges and return a cover containing more vertices than necessary. A smaller cover may exist, but the approximation method does not always find it.

In an event management context, the graph must also represent conflicts correctly. If an edge is missing or an incorrect conflict is included, the resulting cover may not represent the actual scheduling problem. Furthermore, identifying the vertices that cover every conflict edge does not automatically resolve the conflicts. The organizer still needs to decide whether to change an event's time, assign a different venue, or take another action.

The approximation method is therefore useful for identifying a manageable set of conflict-related vertices, but additional scheduling logic and human decisions may be required.

9. Limited Security and Access Control

The level of security provided by the current system depends on the authentication and authorization features actually implemented in the Java program.

A console-based academic application may provide basic access restrictions, but this does not automatically provide the security mechanisms expected from a production system. Sensitive information may require stronger protection, and access to administrative operations must be restricted appropriately. File-based storage can also create security concerns if records are stored without appropriate protection. Users who can directly access the files may be able to read or modify the stored information outside the application.

A more advanced system could include stronger password protection, role-based authorization, secure storage, audit logs, and protection against unauthorized changes. These measures require deliberate implementation and testing; they should not be assumed to exist simply because the application includes a login operation.

10. Absence of Integrated Online Services

The current project focuses on core event management operations and does not include several services commonly found in larger online platforms unless these have been added separately.

For example, the current scope does not include a web frontend, REST API, cloud database, mobile application, real-time multi-user synchronization, online payment processing, or email and SMS notifications.

Without notification services, users may need to check the application manually to obtain event updates. Without payment integration, the system cannot independently process registration fees. Without a shared online service, users cannot automatically access and update the same event records from multiple devices.

These features require additional components, configuration, and testing. They can be considered future enhancements rather than limitations in the basic algorithmic demonstration provided by the present project.

6.3 METHODS FOR REDUCING THE CHALLENGES

Several development practices can help reduce the challenges discussed in this chapter.

First, the program should be divided into clearly defined modules. Each method should have a specific responsibility, and common operations such as reading records or validating input should be implemented consistently.

Second, the system should be tested with different types of input. Testing should include valid records, empty values, incorrect dates, duplicate registrations, missing files, and search queries that produce no results. This helps reveal problems before the application is demonstrated or submitted.

Third, each algorithm should be tested independently with small examples whose answers can be checked manually. For example, KMP can be tested with a pattern that exists in the text and another that does not. Edit Distance can be checked with two identical strings and two strings that differ by one character. Graph algorithms can be tested using small networks with known capacities or simple graphs with easily verified vertex covers.

Fourth, file operations should be checked carefully. The developer should verify that new information is saved, existing information remains available, and records can be retrieved correctly after the application restarts.

Finally, the project report should clearly distinguish between completed functionality and proposed extensions. If an algorithm is described in the report, its implementation should be checked against the Java source code. This ensures that the explanation, pseudocode, complexity analysis, and actual application remain consistent.

7.CONCLUSION AND FUTURE WORK

7.1 Conclusion

The Event Management System is designed to simplify and organize the activities involved in managing events. Event management includes several important tasks, such as maintaining event details, searching for events, registering participants, managing volunteers, recording attendance, and collecting feedback. When these activities are handled manually, it may become difficult to maintain accurate records, retrieve information quickly, and manage all the details in an organized manner. The developed system provides a structured approach to handling these activities through a Java-based application.

The main objective of the project is to demonstrate how programming concepts, data structures, file handling, and algorithms can be applied to solve problems related to event management. The system organizes different operations into separate modules so that each module performs a particular responsibility. This modular approach makes the program easier to understand, test, maintain, and extend. It also reduces the complexity of the application by separating event-related operations from participant management, searching, and data storage.

The Java programming language provides the foundation for implementing the system. Its methods, classes, conditional statements, loops, and exception-handling features can be used to organize the program logic and process user input. The console-based interface allows users to interact with the application through menu options. Users can select the required operation, enter the necessary information, and view the results displayed by the program. Although the interface is simple, it provides a useful way to demonstrate the core logic of an event management application.

File handling plays an important role in maintaining information between program executions. By storing records in files, the application can retrieve previously saved information instead of depending entirely on temporary variables. Event details, registrations, and other records can be maintained according to the file structure used in the implementation. This approach is suitable for an academic project because it demonstrates the basic principles of data persistence without requiring a separate

database server. At the same time, the project highlights the importance of maintaining a consistent data format and handling file operations carefully.

Another important aspect of the project is the study and application of algorithms. Query-Based Event Searching provides a method for locating relevant event records using information entered by the user. The Knuth–Morris–Pratt algorithm demonstrates how a pattern can be searched within a larger text while avoiding unnecessary repeated comparisons. Its use of the Longest Prefix Suffix table helps the algorithm decide where to continue after a mismatch. This provides an efficient approach to exact string matching when compared with straightforward searching methods that may repeat comparisons. Edit Distance introduces a different approach to text processing. Instead of requiring two strings to match exactly, it calculates the minimum number of insertion, deletion, and substitution operations needed to convert one string into another. This technique can help compare event names and search queries that contain small spelling differences. Dynamic Programming is used to store the results of smaller subproblems, allowing the algorithm to calculate the final distance systematically. Through this method, the project demonstrates how an algorithm can measure the similarity between text strings. Hashing provides another approach to processing and comparing information. A hash function converts input data into a numerical value that can be used for filtering or comparison. Hash-based methods can help identify possible matches, but collisions must be considered because different strings can produce the same hash value. Therefore, direct comparison may still be required to confirm a match. Studying hashing helps explain the importance of selecting appropriate data-processing techniques and understanding their limitations.

The project also considers graph-based algorithms for problems involving resource allocation and conflict analysis. The Edmonds–Karp algorithm calculates the maximum flow through a network with defined capacities. When a participant-allocation problem is correctly represented as a flow network, the algorithm can help determine the maximum number of assignments possible under the specified constraints. Similarly, the Vertex Cover Approximation algorithm demonstrates how a set of vertices can be selected so that every edge in a graph has at least one selected endpoint. When graph edges represent conflicts between events, this technique can help identify a group of events that require further examination.

These algorithms address different types of problems and demonstrate the importance of selecting an appropriate solution for each requirement. String-matching algorithms operate on text, Edit Distance compares strings, hashing supports filtering, and graph algorithms handle relationships and constraints. Understanding their time and space complexity also helps explain how their performance may change as the amount of input data increases. The theoretical complexity of an algorithm, however, does not by itself prove that the algorithm has been integrated into the application. Each algorithm described as implemented should be verified against the actual Java source code.

The modular design of the system provides a foundation for further development. Searching, event management, participant registration, attendance, and feedback can be maintained as separate responsibilities while sharing the information required for their operations. This organization makes it easier to identify errors, test individual functions, and introduce additional features without rewriting the entire application.

The current implementation also has certain limitations. A console interface does not provide the visual convenience of a graphical web or mobile application. File-based storage is straightforward for a small project but may become less suitable when the number of records grows or several users need to update information simultaneously. Advanced facilities such as online payments, automated notifications, cloud-based access, and real-time scheduling require additional components and are not provided automatically by the basic system.

Despite these limitations, the Event Management System provides a useful academic foundation for understanding how Java programming and algorithmic techniques can be applied to event-related operations. It combines fundamental software development concepts with examples of text processing, data storage, and graph-based problem-solving. The project also demonstrates the importance of modularity, input validation, efficient searching, and appropriate data management.

In conclusion, the project establishes a structured approach to managing event information and studying the algorithms that can support more advanced event-related operations. Its present design can serve as a starting point for developing a more complete system with improved usability, stronger storage facilities, and additional scheduling and management capabilities. The project also provides practical learning experience in translating problem statements into program logic, organizing code into modules, testing algorithmic solutions, and evaluating the efficiency of different approaches.

7.2 FUTURE WORK

Although the current Event Management System provides a foundation for managing event information, several improvements can be introduced to make it more efficient, user-friendly, secure, and suitable for real-world use. Future development can focus on improving the interface, strengthening data storage, supporting multiple users, enhancing searching, and extending the scheduling and resource-allocation capabilities. Future enhancements could investigate improved string-searching methods and capacity-constrained allocation techniques [4], [5].

1.Development of a Graphical User Interface

One of the major improvements that can be introduced in the future is the development of a graphical user interface. The present console-based interface requires users to interact through text-based menus and keyboard input. While this approach is suitable for demonstrating program logic, it may not provide the most convenient experience for users who manage many events.

A graphical interface could present event information using organized tables, cards, forms, and dashboards. Users could create events through forms containing fields for the event name, date, time, venue, and description. Participants could view available events and access registration options without navigating through several text-based menus.

The interface could also include a calendar view that displays events according to their scheduled dates. Visual indicators could be used to distinguish upcoming events, completed events, and events that require attention. Search boxes and filter controls could help users find information more conveniently.

Such an improvement would make the system easier to use and would provide a more modern presentation. However, developing the interface would require additional frontend code and integration with the existing Java application or a suitable backend service.

2.Integration with a Database Management System

Another important future improvement is replacing or extending file-based storage with a database management system. Text files are useful for a small academic project, but managing a large number of related records through files can become difficult.

A database such as MySQL could store event details, user information, participant registrations, volunteer assignments, attendance, and feedback in separate but related tables. Each table could have a defined structure, and relationships could be established using primary keys and foreign keys.

For example, an event table could store event identifiers, names, dates, times, and venues. A registration table could store the participant identifier and the event identifier. This would make it easier to retrieve all participants registered for a particular event or display all events associated with a participant.

Database queries could also improve the retrieval of records based on different conditions. Organizers could search by event date, venue, category, or registration status without manually reading every file. Database constraints and transactions could help maintain consistency when related records are updated.

The integration of a database would therefore improve the system's ability to handle larger amounts of information. It would also provide a stronger foundation for multi-user access and more advanced reporting.

3.Improvement of Event Searching

The searching module can be extended to provide more flexible and accurate results. A basic search may identify events only when their names or descriptions contain the entered keyword. However, users may enter spelling mistakes, abbreviations, or different words that express a similar interest.

In the future, the system could combine exact string matching with approximate matching. KMP could be used for exact pattern searches, while Edit Distance could measure how closely a search query resembles an event name. The system could then rank possible matches according to an appropriate similarity rule.

Additional filters could be introduced for event category, date, location, organizer, and availability. For example, a user interested in sports activities could filter events by category and select a particular date range. Combining filters with keyword searching would make the search process more useful.

The system could also display suggestions when no exact result is found. If the user enters a slightly incorrect event name, the application could display similar event names that fall within a selected edit-distance threshold. The threshold should be chosen carefully because an excessively strict value may miss useful results, while an excessively relaxed value may return unrelated events.

Further improvements could include ranking results by relevance and storing frequently used search information where appropriate. These changes would make event discovery more convenient as the amount of event data increases.

4.Enhancement of Hash-Based Searching

Hashing can be improved to support more efficient processing of large amounts of text. A future implementation could use an appropriate rolling-hash algorithm to calculate and update hash values efficiently while searching through a text.

Randomized hashing could also be studied and implemented where the chosen method benefits from randomly selected parameters. The implementation should clearly define how parameters are selected and how collisions are handled.

A major requirement is ensuring correctness. Hash values should be used to identify possible matches rather than automatically proving that two strings are identical. When necessary, the system should compare the actual strings to verify the result.

Future work could compare the performance of straightforward string searching, KMP, and hash-based searching using event descriptions of different lengths. The results could be measured using execution time and memory consumption. This would provide a clearer understanding of when each method is suitable.

5. Advanced Participant Allocation

The participant-management module could be extended to handle event capacity and assignment constraints more effectively. In a larger system, an event may have a limited number of places, and participants may be eligible for different events according to their interests or requirements.

A future version could model participants and events as a flow network. The source could connect to participant nodes, participant nodes could connect to eligible event nodes, and event nodes could connect to the sink with capacities representing the available places.

The Edmonds–Karp algorithm could then calculate the maximum flow through the network. Under a correctly defined model, this value would represent the maximum number of assignments possible while respecting the specified capacities.

Further development could introduce participant preferences, priority rules, time restrictions, and eligibility requirements. These conditions must be represented appropriately in the model or handled by additional optimization logic. Standard maximum flow alone does not automatically maximize participant satisfaction or produce the best possible timetable.

The system could also display the allocation results to the organizer, including assigned participants and events that have reached their capacity. Such information would support more organized planning and help reduce manual work.

6. Improved Venue and Schedule Conflict Detection

Another possible enhancement is an event-conflict analysis module. When several events are scheduled for the same period, they may require the same venue, equipment, or other limited resources. These conflicts can create difficulties for organizers if they are not identified early.

A future version could represent events as vertices in a graph. An edge could connect two events when they have a conflict, such as overlapping time periods at the same venue. The system could examine this graph and identify groups of events that require attention.

Vertex Cover Approximation could be used to select a set of vertices that covers all conflict edges. This would provide a group of events that collectively touches every represented conflict. The organizer could then examine those events and decide whether to reschedule them or assign different venues.

The system could also be extended with time-overlap checking, venue availability information, and resource constraints. These additional rules would help identify conflicts more accurately. The graph model must be built from the actual event dates, times, venues, and resource requirements for its results to be meaningful.

This improvement could reduce scheduling errors and help organizers identify potential problems before events take place.

7. Multi-User Access and Role-Based Authorization

The current project can be extended to support different types of users with clearly defined permissions. For example, organizers may be allowed to create and modify events, participants may be allowed to register and submit feedback, and volunteers may need access to assigned duties and attendance-related information.

A future version could provide a shared backend service through which users access the same stored information. This would allow several users to work with the application from different devices, provided that the backend and storage system are configured appropriately.

Role-based authorization could ensure that each user can perform only the operations permitted for their role. For example, a participant should not be able to modify an event schedule unless the application explicitly grants that permission.

The system could also maintain audit records of important actions, such as creating an event, changing a venue, or modifying a registration. These records would help administrators review changes and investigate unexpected behaviour.

To support reliable simultaneous access, the application would need appropriate database transactions and concurrency controls. These improvements would make the system more suitable for environments in which many users interact with event records at the same time.

8.Security Improvements

Security can be strengthened by introducing additional protections for user accounts and stored information. A future version could use secure password hashing, input validation, role-based authorization, and appropriate session-management mechanisms.

Sensitive information should not be stored in plain text unless there is a clear and justified reason. Access to stored records should also be controlled so that unauthorized users cannot easily read or modify them.

The application could validate user input before processing it and apply suitable protections to prevent unauthorized operations. Error messages should provide useful information without unnecessarily exposing internal implementation details.

If the system is connected to a network, secure communication and suitable backend access controls would also be important. Security testing should be performed to identify weaknesses in authentication, authorization, data storage, and input processing.

9.Automatic Notifications and Reminders

A future version could include automatic notifications to inform users about upcoming events and important schedule changes. At present, users may need to open the application manually to check event information, depending on the features available in the current implementation.

Notification functionality could send reminders before an event begins. Participants could also receive updates when the date, time, venue, or registration status changes.

For example, a participant registered for a workshop could receive a reminder one day before the event. If the venue changes, the system could notify the registered participants so that they do not arrive at the wrong location.

Notifications could be delivered through email, SMS, or application-based messages, depending on the services integrated into the system. The application would need to maintain appropriate contact information and handle notification failures carefully.

Such a feature would improve communication between organizers, participants, and volunteers. It could also reduce the possibility of missed events and confusion caused by schedule changes.

10.Online Registration and Payment Integration

The system could be extended to support online registration and payment processing if events require registration fees. Users could view event details, register online, and receive confirmation after completing the required steps.

A payment gateway could be integrated to process payments securely. The application would need to verify payment status before marking a paid registration as complete. It should also handle failed payments, repeated requests, and cancellations correctly.

Registration limits could be applied so that the system does not accept more participants than an event can accommodate. The application could display the number of available places and update the registration status after a successful transaction.

This feature would be particularly useful for workshops, conferences, competitions, and other events that require paid registration. However, payment integration would introduce additional security, reliability, and testing requirements and would need to use an appropriate payment service.

11.Performance Testing and Optimization

As the amount of event information increases, the performance of the application should be evaluated systematically. Future work could include testing the system with different numbers of events, participants, registrations, and feedback records.

Searching algorithms could be compared using inputs of different lengths. The execution time of KMP, straightforward searching, and hash-based methods could be measured to understand their practical differences.

Edit Distance could be tested with strings of different lengths to observe how the size of its dynamic programming table affects execution time and memory usage. Graph algorithms could be tested with networks containing different numbers of vertices, edges, and capacity constraints.

The results could be used to identify operations that require optimization. For example, repeated file reading may be reduced by loading data at appropriate times, while database indexing could improve retrieval when a database is introduced.

Performance testing would help determine whether the chosen algorithms and data structures remain suitable as the application grows. It would also provide evidence to support decisions about future improvements.

12. Reporting and Analytics

A further improvement is the addition of reporting and analytics facilities. Organizers may need summarized information about the number of events, registrations, attendees, volunteers, and feedback records.

The application could generate reports showing participant counts for individual events, attendance percentages, registration totals, and feedback summaries. Such reports could help organizers understand how successfully events were conducted.

Data could also be exported into formats such as CSV for further analysis using spreadsheet software. If a database is introduced, queries could be used to generate reports for selected dates, event categories, or venues.

Over time, the system could analyze historical event records to identify popular event categories, attendance patterns, and frequently reported problems. Such analysis could help organizers plan future activities more effectively.

Any conclusions generated from the data should be based on accurate records and clearly defined calculations. Analytics would support planning, but it would not automatically guarantee that future events will achieve the same results.

13.Mobile and Cloud-Based Access

The application could eventually be extended to support mobile devices and cloud-based access. A mobile-friendly interface would allow participants and volunteers to check event details, register for activities, and receive updates without relying on a desktop computer.

Cloud-based storage could make the same event information available to authorized users from different locations. A shared backend would manage access to the data and provide a common source of information for the different clients.

This approach would require suitable authentication, authorization, secure communication, database management, and deployment procedures. The system would also need to handle network interruptions and failed requests appropriately.

A mobile or cloud-based version would expand the accessibility of the project and make it more suitable for larger events. However, it would require additional development and testing beyond the present console-based implementation.

7.3 EXPECTED OUTCOME OF FUTURE ENHANCEMENTS

The proposed future improvements are intended to make the Event Management System more practical and efficient as its requirements grow. A graphical interface could improve usability, database integration could strengthen record management, and improved search methods could help users locate events more easily.

Graph-based allocation and conflict-analysis features could assist organizers in managing limited resources and identifying scheduling problems. Role-based access controls and stronger security measures could protect important records, while notification services could improve communication with participants and volunteers.

Performance testing would help determine which parts of the application need optimization. Reporting and analytics could provide useful summaries of event activity, and mobile or cloud-based access could make the application available to a wider range of users.

These improvements are proposed extensions rather than guaranteed features of the current project. Each enhancement would need to be designed, implemented, tested, and evaluated separately before being considered part of the completed system.

REFERENCES

[1] D. E. Knuth, J. H. Morris, Jr., and V. R. Pratt, “Fast Pattern Matching in Strings,” *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, 1977. https://doi.org/10.1137/0206024.

[2] V. I. Levenshtein, “Binary Codes Capable of Correcting Deletions, Insertions, and Reversals,” *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[3] R. A. Wagner and M. J. Fischer, “The String-to-String Correction Problem,” *Journal of the ACM*, vol. 21, no. 1, pp. 168–173, 1974. https://doi.org/10.1145/321796.321811.

[4] R. M. Karp and M. O. Rabin, “Efficient Randomized Pattern-Matching Algorithms,” *IBM Journal of Research and Development*, vol. 31, no. 2, pp. 249–260, 1987. https://doi.org/10.1147/rd.312.0249.

[5] J. Edmonds and R. M. Karp, “Theoretical Improvements in Algorithmic Efficiency for Network Flow Problems,” *Journal of the ACM*, vol. 19, no. 2, pp. 248–264, 1972. https://doi.org/10.1145/321694.321699.

[6] R. M. Karp, “Reducibility Among Combinatorial Problems,” in *Complexity of Computer Computations*, pp. 85–103, 1972. https://doi.org/10.1007/978-1-4684-2001-2_9.

[7] R. Bar-Yehuda and S. Even, “A Local-Ratio Theorem for Approximating the Weighted Vertex Cover Problem,” in *Proceedings of the 2nd Scandinavian Workshop on Algorithm Theory*, 1985.

[8] F. J. Damerau, “A Technique for Computer Detection and Correction of Spelling Errors,” *Communications of the ACM*, vol. 7, no. 3, pp. 171–176, 1964. https://doi.org/10.1145/363958.363994.

[9] S. B. Needleman and C. D. Wunsch, “A General Method Applicable to the Search for Similarities in the Amino Acid Sequence of Two Proteins,” *Journal of Molecular Biology*, vol. 48, no. 3, pp. 443–453, 1970. https://doi.org/10.1016/0022-2836(70)90057-4.

[10] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA: MIT Press, 2022.